

Ontwikkeling van een smartphone SDK voor het obfusceren van locatie data

Milan SCHOLLIER

Promotor(en): prof. dr. Vincent Naessen Masterproef ingediend tot het behalen van
de graad van master of Science in de
Co-promotor(en): dr. ing. Michiel Willocx industriële wetenschappen: Master in de
industriële wetenschappen elektronica-ICT:
smart applications

Academiejaar 2024 - 2025

©Copyright KU Leuven

Deze masterproef is een examendocument dat niet werd gecorrigeerd voor eventuele vastgestelde fouten.

Zonder voorafgaande schriftelijke toestemming van zowel de promotor(en) als de auteur(s) is overnemen, kopiëren, gebruiken of realiseren van deze uitgave of gedeelten ervan verboden. Voor aanvragen i.v.m. het overnemen en/of gebruik en/of realisatie van gedeelten uit deze publicatie, kan u zich richten tot KU Leuven Technologicampus Gent, Gebroeders De Smetstraat 1, B-9000 Gent, +32 92 65 86 10 of via e-mail iiw.gent@kuleuven.be.

Voorafgaande schriftelijke toestemming van de promotor(en) is eveneens vereist voor het aanwenden van de in deze masterproef beschreven (originele) methoden, producten, schakelingen en programma's voor industrieel of commercieel nut en voor de inzending van deze publicatie ter deelname aan wetenschappelijke prijzen of wedstrijden.

Dankwoord

Voor u ligt de thesis "Ontwikkeling van een smartphone SDK voor het obfusceren van locatie data". Deze thesis is geschreven om te voldoen aan de afstudeereisen van de Master in de industriële wetenschappen elektronica-ICT: smart applications aan de Katholieke Universiteit van Leuven.

Tijdens het schrijven van deze thesis heb ik veel geleerd over zowel de technische aspecten van locatie-obfuscatie als over het zelfstandig uitvoeren van onderzoek. Het was een uitdagend proces waarin ik mijn kennis over softwareontwikkeling, privacy en mobiele applicaties heb kunnen verdiepen. Zeker de diepgang in de werking van het Android besturingssysteem was een uiterst interessant onderwerp, verder waren de experimenten in deze thesis uniek doordat ik buiten kon gaan wandelen, fietsen,... en hieruit rechtstreeks enorm visuele resultaten kon bekomen. Dit zorgde voor directe validatie en motivatie.

Graag maak ik van deze gelegenheid gebruik om enkele mensen te bedanken die een belangrijke rol hebben gespeeld bij het tot stand komen van deze thesis.

In de eerste plaats wil ik mijn promotor, prof. dr. Vincent Naessens, bedanken voor het vertrouwen in dit onderzoek en voor het aanreiken van dit boeiende onderwerp. Mijn oprechte dank gaat ook uit naar mijn masterproefbegeleider, dr. ing. Michiel Willocx, voor de deskundige begeleiding, waardevolle feedback en voortdurende motivatie gedurende het hele traject. Daarnaast wil ik de medewerkers van Distring bedanken voor hun steun en de aangename gesprekken doorheen het jaar.

Verder ben ik mijn vrienden en familie, Bran, Cas, Melissa, Robin, Warre, 2x Lotte, Angelo, Elise, Sofie en vele anderen dankbaar voor hun vriendschap, aanmoediging en de mooie jaren samen in het hoger onderwijs.

Tot slot wil ik iedereen bedanken die mij op welke manier dan ook heeft gesteund tijdens mijn masterjaar. Jullie steun heeft het verschil gemaakt.

Ik hoop dat het onderwerp u even hard kan boeien als mij.

Milan Schollier

Gent, May 22, 2025



*Copyright informatie:
studiewerk van een student in het kader van de
academische opleiding en examenbeoordeling.
Na deze beoordeling vond geen correctie plaats
van het studiewerk.*

Abstract

0.1 Abstract

In today's data-driven economy, location data have enormous value. It is used to gain valuable insights about crowds and to optimize various sectors of society, ranging from tourism to retail.

However, the methods used to collect this valuable location data are often done without the explicit, informed consent of the user. The high precision of location data and the frequency of reporting pose significant privacy risks for individuals.

These risks include not only the inference of sensitive locations, such as residence, workplace and leisure activities, but also the construction of social graphs. In the hands of malicious actors, this information can lead to blackmail, coercion or hate crimes.

To address these privacy concerns, this thesis proposes a privacy-friendly alternative to collecting valuable location data. An innovative Android library is developed that obfuscates both locations and timestamps by adding noise, making individual traces no longer linkable to a specific user. This prevents sensitive information from leaking, while preserving the usability of the data. Importantly, the library runs entirely on the device, ensuring that the real, accurate data never leaves the device.

To achieve this, this thesis explores the Android ecosystem, including lifecycles and background tasks, to design and implement advanced location obfuscation algorithms within the library. A series of experiments are conducted to demonstrate the effectiveness of the solution. The thesis compares the proposed library with the Approximate Location feature introduced in Android 12, and provides insights into the advancements in mobile privacy measures.

0.2 Samenvatting

In de huidige data-gedreven economie heeft locatiedata een enorme waarde. Het wordt gebruikt om waardevolle inzichten te verkrijgen over menigten en om verschillende sectoren van de samenleving te optimaliseren, variërend van toerisme tot retail.

De methoden die worden gebruikt om deze waardevolle locatiegegevens te verzamelen, worden echter vaak uitgevoerd zonder de expliciete, geïnformeerde toestemming van de gebruiker.

De hoge precisie van locatiedata en de frequentie van rapportages brengen aanzienlijke privacyrisico's met zich mee voor individuen.

Deze risico's omvatten niet alleen het afleiden van gevoelige locaties, zoals woonplaats, werkplek en vrijetijdsactiviteiten, maar ook de constructie van sociale grafen. In handen van kwaadwillende actoren kan deze informatie leiden tot chantage, dwang of haatmisdrijven.

Om deze privacyproblemen aan te pakken, stelt deze thesis een privacyvriendelijk alternatief voor, voor het verzamelen van waardevolle locatiegegevens voor. Een innovatieve Android-library is ontwikkeld die zowel locaties als tijdstempels obfusceert door ruis toe te voegen, waardoor individuele sporen niet langer aan een specifieke gebruiker kunnen worden gekoppeld. Dit voorkomt dat gevoelige informatie uitlekt, terwijl de bruikbaarheid van de data behouden blijft. Belangrijk is dat de library volledig op het apparaat werkt, waardoor de echte, nauwkeurige gegevens het apparaat nooit verlaten.

Om dit te bereiken, onderzoekt deze thesis het Android-ecosysteem, inclusief levenscycli en achtergrondtaken, om geavanceerde algoritmes voor locatie-obfuscatie te ontwerpen en te implementeren binnen de library. Een reeks experimenten is uitgevoerd om de effectiviteit van de oplossing aan te tonen. De thesis vergelijkt de voorgestelde library met de Approximate Location-functie die is geïntroduceerd in Android 12, en biedt inzichten in de vooruitgang op het gebied van mobiele privacymaatregelen.

Trefwoorden: Android library, location obfuscation, location tracking

Contents

0.1	Abstract	vii
0.2	Samenvatting	vii
1	Inleiding	3
1.1	Probleemstelling	4
1.2	Onderzoeksvragen	5
1.3	Doelen en scope	5
1.4	Overzicht van de thesis	6
2	State of the art	7
2.1	Android besturingssysteem	7
2.1.1	Library	7
2.1.2	Activity	8
2.1.3	Achtergrondwerk	8
2.2	Locatie obfuscatie	11
2.2.1	Android's oplossing	12
2.2.2	Voorgaand onderzoek	12
2.3	Preliminare experimenten	14
2.3.1	Verschillende nauwkeurigheidsniveaus	14
2.3.2	Coarse location Android	15
3	Design	19
3.1	Vereisten	19
3.2	Veronderstellingen	20
3.3	Ontwerp	21
4	Implementatie	23
4.1	Overzicht library	23
4.1.1	Locatie tracking	23

4.1.2	Obfuscatie	28
4.1.3	Opslag	30
4.1.4	Netwerk	30
4.2	Applicatie	31
4.3	Dataverwerking	32
4.3.1	Verzameling	32
4.3.2	Verwerking	33
4.4	Conclusie	33
5	Experimenten en resultaten	35
5.1	Visuele representatie	35
5.1.1	Variabelen in kaart brengen	36
5.1.2	Uiteindelijke resultaat	37
6	Evaluatie en discussie	39
6.1	Evaluatie van de onderzoeksvragen	39
6.1.1	Gebruiksvriendelijkheid en uitbreidbaarheid	39
6.1.2	Effectiviteit van de obfuscatie	39
6.1.3	Beperkingen	41
6.2	Verder werk	41
6.3	Gebruik van de library	42
6.3.1	Initialisatie	42
6.3.2	LocationObfuscator zelf implementeren	42
6.3.3	Live data	43
7	Conclusie	45
A	Code van het obfuscatie algoritme	49
B	Code van de evaluatie functie	51
C	Beschrijving van deze masterproef in de vorm van een wetenschappelijk artikel	53
D	Poster	57
E	GenAI code of conduct for students	59

List of Figures

2.1	Activity lifecycle Android[5]	9
2.2	Keuze boom achtergrondwerk voor event driven tasks [6]	10
2.3	Android WorkManager	11
2.4	Vergelijking coarse permissies en fine permissies 1	16
2.5	Vergelijking coarse permissies en fine permissies 2	17
2.6	Vergelijking coarse permissie op één apparaat	17
2.7	Vergelijking coarse permissie op meerdere apparaten	17
3.1	Flow diagram van continue locatieverzameling	21
4.1	Klasse diagram van de library	24
4.2	Flow diagram van het obfuscatie algoritme	30
4.3	Visualisatie van de test applicatie	32
5.1	Echte pad	36
5.2	Vergelijking grootte privacycirkel	37
5.3	Vergelijking tijdsinterval locatiegegevens	37
5.4	Obfuscatie-algoritme gevisualiseerd	38
6.1	Kwantitatieve evaluatieresultaten	40

Acronyms

AAR Android archive. 7

API application program interface. 8, 9, 11, 12, 21, 30

APK Android package. 7, 27

GPS Global Positioning System. 12, 13, 14, 33, 35

1

Inleiding

Veel moderne applicaties verzamelen locatiegegevens van hun gebruikers. Deze gegevens worden vaak gebruikt voor crowd-insights die nuttig zijn voor uiteenlopende doeleinden. Voorbeelden van onderzoeken die hierdoor mogelijk gemaakt worden zijn mobiliteitsstudies en stadsplanning. Een voorbeeld hiervan is een winkelketen die wil bepalen waar een nieuwe vestiging het best kan worden gebouwd, zonder bestaande locaties te hinderen. Zo is Gravy analytics een data broker die data opkoopt en dan analyses of marktonderzoeken aanbiedt aan bedrijven. Echter zorgt hoge nauwkeurigheid en frequentie waarmee locatiegegevens worden verzameld, voor aanzienlijke privacyrisico's met zich mee. Zo zijn er altijd wel datalekken, zoals het incident bij Gravy Analytics [1]. Waarbij locatiegegevens van miljoenen gebruikers werden gelekt na een inbraak. In verkeerde handen kunnen dergelijke gegevens leiden tot sociale profilering of zelfs spionage, wat de persoonlijke veiligheid van gebruikers in gevaar brengt.

Bovendien is een datalek niet altijd nodig om privacyproblemen te veroorzaken. Onzorgvuldig omgaan met afgeleide gegevens kan eveneens risico's opleveren. Zo bleek bij datingapps dat gebruikers konden worden getrianguleerd via de functie die de afstand tot andere personen toont [2].

Door deze risico's worden zowel het publiek als overheden steeds voorzichtiger. Een voorbeeld hiervan is de overname van de populaire datingapp Grindr door een Chinees bedrijf, waarbij zorgen rezen over de omgang met locatiegegevens. Het Amerikaanse leger heeft het gebruik van deze app, die locatiegegevens vereist, zelfs als veiligheidsrisico bestempeld, omdat zo de locatie van soldaten en militaire bases bekend zou kunnen raken [3].

Met deze thesis wordt een bijdrage geleverd aan de ontwikkeling van privacyvriendelijke tech-

nologieën voor mobiele toepassingen. De voorgestelde oplossing zorgt ervoor dat gevoelige locatiedata zelfs nooit het apparaat verlaat waardoor vroeger in de ketting naar de data broker het probleem wordt aangepakt. Ook wilt deze thesis de instapdrempel voor ontwikkelaars drukken en de gebruiksvriendelijkheid bevorderen zodat veilig met locatiedata omgaan een standaard wordt in een wereld waarin data steeds waardevoller — en kwetsbaarder — worden.

1.1 Probleemstelling

Het centrale probleem is dat de werkelijke locatie van een gebruiker doorgaans rechtstreeks naar de server wordt verzonden en daar zonder enige vorm van obfuscatie wordt opgeslagen. Hierdoor blijven locatiegegevens in hun oorspronkelijke, niet-versluierde vorm beschikbaar, wat aanzienlijke privacyrisico's met zich meebrengt. Bovendien worden deze gegevens vaak zonder verdere bewerking doorverkocht aan data brokers.

In de huidige stand van zaken is privacy moeilijk te waarborgen. namelijk in de huidige stand van zaken om privacy te garanderen moet de ketting van apparaat naar data broker er als volgt uit zien.

- In de overdracht van de data naar de server moet volledig beveiligd zijn. Zo mag het netwerkverkeer geen slachtoffer zijn van gekende zwaktes zoals man-in-the-middle attack waar een hacker al het netwerkverkeer kan afluisteren.
- Eenmaal de data toekomt bij de server moet de locatiedata versluierd worden om bij eventuele datalekken de gebruikers te beschermen.
- Bij het gebruik van deze data, moest het nog niet versluierd zijn, moet ook veiligheid in achtgenomen worden zodat je geen triangulatie krijgt zoals bij dating apps vroeger. Want in deze stap kunnen derde partijen ook gevoelige data verzamelen.
- Dan als deze data wordt verkocht aan een data broker, en de data zou nog steeds niet versluierd zijn, moet dit zeker versluierd doorverkocht worden.
- Naast al deze veiligheidsrisico is een sociaal veiligheidsrisico waarbij de data broker geen goede intenties heeft ook van belang. Waardoor een strikte controle op de partijen die toegang krijgen tot deze gegevens noodzakelijk om misbruik te voorkomen.

De aanwezigheid van deze potentiële kwetsbaarheden bemoeilijkt het voor gebruikers om vertrouwen te hebben in de huidige verwerking van locatiegegevens.

Naast dit centraal probleem zorgen de steile leercurve van de relevante Android-technologieën en vaak voorkomende strakke deadlines voor een bijkomende uitdaging. Deze bijkomende uitdagingen kunnen hierdoor zorgen voor beveiligingslacunes in de implementatie. Een goed ontworpen library die de complexiteit voor ontwikkelaars verlaagt en standaard voorziet in robuuste beveiligingsmaatregelen, kan hierdoor deze risico's aanzienlijk beperken. Dit is

namelijk de mentaliteit dat de framework Django handhaaft “Django makes it easier to build better web apps more quickly and with less code.” en “Reassuringly secure. Django takes security seriously and helps developers avoid many common security mistakes.”. [4]

1.2 Onderzoeksvragen

Het onderzoek beantwoordt 2 kernvragen: Hoe kan een Android-library worden ontworpen die locatiegegevens lokaal en gebruiksvriendelijk obfusceert?

- Is het mogelijk voor een duidelijke abstractie waardoor de developer ook eigen obfuscatie algoritmes makkelijk kan implementeren?
- Is de standaard initialisatie zo kort als mogelijk? Met goed gekozen standaard instellingen.
- Hoe makkelijk is de library te implementeren in een bestaand project waar locatie verzameling aan toegevoegd zou worden?

In hoeverre beschermd de obfuscatie de locatie van de gebruiker?

- Kan een algemeen pad, dat nog nuttig is voor marktonderzoek, nog afgeleid worden uit het geobfusceerde pad?
- Is de data wel degelijk veilig in de library?
- In hoeverre is geobfusceerde data nog nuttig voor data brokers?

1.3 Doelen en scope

Het doel van deze thesis is om een location data stream obfuscator te implementeren als een Android-library. Waarbij de obfuscator locatiegegevens lokaal versluiert en dat de echte gegevens nooit het apparaat verlaten. Dit zodat individuele bewegingspatronen niet meer naar specifieke personen kunnen worden herleid. De ontwikkeling van deze library zou de obfuscatie berekening van de server naar de client verhuizen. Waardoor gevoelige informatie (bv. een privacy bubbel) lokaal op het apparaat blijft en enkel geobfusceerde locaties worden doorgestuurd en zo data lekken minder schadelijk zijn.

In deze thesis wordt de focus uitsluitend gelegd op het lokaal verzamelen en obfusceren van locatie- en tijdgegevens op Android-apparaten. Aspecten zoals server-side beveiliging, netwerkbeveiliging en bredere privacyvraagstukken, waaronder identiteitsbeheer en gebruiker-sconsent, worden buiten beschouwing gelaten. De implementatie is beperkt tot het Android-platform; andere mobiele besturingssystemen, zoals iOS, vallen buiten de scope van dit onderzoek.

1.4 Overzicht van de thesis

Hoofdstuk 2 State of the art: In dit hoofdstuk wordt een overzicht gegeven van de relevante aspecten van het Android besturingssysteem die van belang zijn voor deze thesis. Daarnaast wordt de huidige stand van zaken omtrent locatieverzameling en obfuscatie besproken. Ook worden enkele preliminaire experimenten toegelicht die bijdragen aan een beter begrip van de implementatie in Android, wat de basis vormt voor de keuzes die in Hoofdstuk 4 worden gemaakt.

Hoofdstuk 3 Design: In dit hoofdstuk worden de vereisten en veronderstellingen van de library besproken. Daarnaast worden de belangrijkste ontwerpkeuzes en architecturale beslissingen toegelicht die aan de basis liggen van de implementatie. Dit doordat de limitaties of verplichtingen in het Android-systeem enorm bepalend zijn voor dit.

Hoofdsuk 4 Implementatie: In dit hoofdstuk wordt de volledige werkwijze van implementatie toegelicht. Hier wordt een overzicht van de library besproken waar functionaliteit per functionaliteit besproken wordt hoe dit bereikt is geweest. Daarna wordt visueel besproken hoe de testapplicatie in elkaar zat om de experimenten te kunnen voltooien.

Hoofdsuk 5 Resultaten: In dit hoofdstuk wordt eerst toegelicht hoe de experimenten voor de uiteindelijke library zullen worden uitgevoerd worden. Dan wordt een vergelijking gemaakt in hoe de parameters in de standaardimplementatie van de locatieobfuscator het uiteindelijke resultaat beïnvloedt. Om dan af te sluiten met een totaal plaat van de werking van de library.

Hoofdsuk 6 Evaluatie en discussie: In dit hoofdstuk kijken we of de library voldoet aan de onderzoeksvragen en andere sterktes waar de resulterend library gebruikt kan worden. Ook worden de beperkingen en eventueel verder onderzoek besproken. Om dan af te sluiten met een handleiding van hoe de library te gebruiken is.

2

State of the art

Om de achtergrond te schetsen waarbinnen het werk van deze thesis besproken kan worden, onderscheiden we twee grote stromingen: het Android besturingssysteem en locatieverzameling/-obfuscatie. Ook is er een extra sectie voorzien voor het onderzoek naar hoe de standaardimplementatie van obfuscatie werkt in Android want de documentatie bespreekt sommige dingen niet.

2.1 Android besturingssysteem

2.1.1 Library

Omdat ontwikkelaars een code efficiënt zouden kunnen implementeren zonder herhaaldelijk gebruik te maken van copy-paste, worden libraries gebruikt. Android-applicaties worden voornamelijk geprogrammeerd in Java of Kotlin, waarbij de build-tool Gradle wordt ingezet. Deze build-tool beheert de installatie van de benodigde libraries, zodat deze eenvoudig beschikbaar zijn voor integratie in de eigen code. In tegenstelling tot een standaard applicatie, die wordt gecompileerd naar een Android package (APK), wordt een library voor Android gecompileerd naar een Android archive (AAR).

Deze AAR-bestanden kunnen vervolgens worden gepubliceerd naar publieke Gradle-plugins, waardoor ze eenvoudig herbruikbaar zijn in andere projecten door deze te importeren.

2.1.2 Activity

Een activity in Android is een enkele, gefocuste taak die een gebruiker kan uitvoeren. Het vormt een van de fundamentele bouwstenen van een Android-applicatie. Elke activity heeft een eigen lifecycle, die wordt beheerd door het Android-systeem. Deze lifecycle bestaat uit verschillende states.

Elke state-overgang heeft ook een methode zoals `onCreate`, `onStart`, `onResume`, `onPause`, `onStop` en `onDestroy`. Die de ontwikkelaar kan overschrijven om zo de overgang beter te kunnen afhandelen.

Bij het ontwikkelen van een applicatie is het belangrijk om rekening te houden met deze lifecycle-methoden, omdat ze bepalen hoe de applicatie reageert op gebruikersinteracties en systeemgebeurtenissen. Bijvoorbeeld, in de `onCreate`-methode wordt de initiële setup van de activity uitgevoerd, zoals het instellen van de gebruikersinterface. De `onPause`- en `onStop`-methoden worden aangeroepen wanneer de activity naar de achtergrond gaat, en resources kunnen hier worden vrijgegeven om geheugen te besparen.

Zo heeft elke state bepaalde eigenschappen. Zo worden langdurige taken die in de states draaien, waarbij de activity naar de achtergrond gaat, abrupt gestopt als het systeem resources nodig heeft en is dus niet betrouwbaar. Om dit probleem te omzeilen biedt Android alternatieve oplossingen zoals `services` (waarover sectie hoofdstuk meer). Daarnaast is het niet toegestaan om blocking calls, zoals een HTTP-request of het opvragen van locatiegegevens via de Location Provider, uit te voeren in de main thread. Dit is noodzakelijk om te voorkomen dat de gebruikersinterface vastloopt en de gebruikservaring negatief wordt beïnvloed. Zo crashed de applicatie als de main thread wordt geblocked voor meer dan vijf seconden.

2.1.3 Achtergrondwerk

Om de hierboven vermelde beperkingen van een activity, die slechts kort op de achtergrond actief kan blijven, te omzeilen. Biedt Android verschillende tools die door de jaren heen zijn geëvolueerd. Deze tools maken het mogelijk om taken los van de activity uit te voeren, zelfs wanneer de applicatie niet actief is. Maar elke application program interface (API) is verschillend om zo elk soort probleem aan te pakken. Hier focussen we op de keuze boom gerelateerd voor het starten van een achtergrond taak gebaseerd op een event. Dit omdat de optie om een eenmalige locatie op te vragen in de applicatie kan afgehandeld worden door een simpele thread.

Zoals te zien in Figuur 2.2 is asynchrone werk niet van toepassing voor deze thesis omdat het over langdurige verzameling gaat. Dan heb je nog Task scheduling en Foreground services. Hierin zijn de belangrijkste tools zijn de `AlarmManager` en de `WorkManager`.

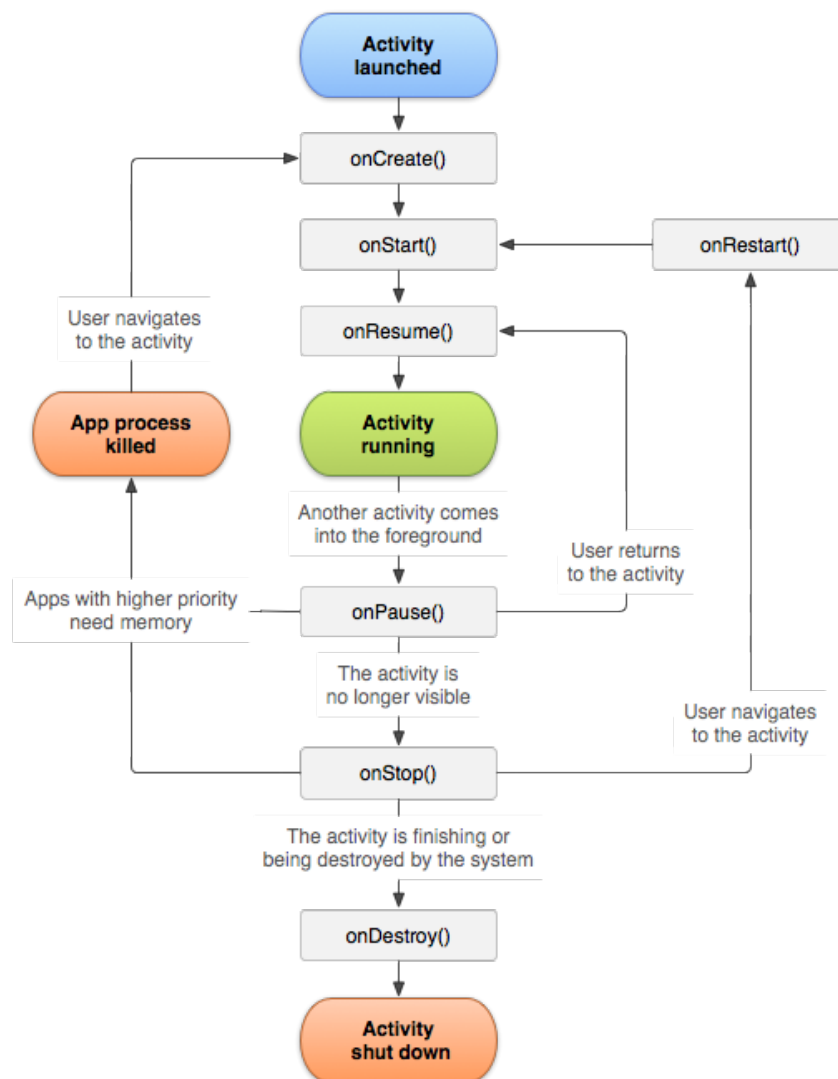


Figure 2.1: Activity lifecycle Android[5]

AlarmManager [7] Alarm manager is een implementatie van een task scheduling API. De AlarmManager is een mechanisme dat het apparaat uit de zogenaamde 'doze mode' of energiebesparende modus kan halen. Dit maakt het mogelijk om op specifieke tijdstippen een taak uit te voeren. Echter, de AlarmManager heeft enkele beperkingen. Zo kan het slechts één keer per uur een taak uitvoeren en is het niet geschikt voor langdurige taken. Dit maakt het minder geschikt voor toepassingen die continu of frequent achtergrondtaken moeten uitvoeren. AlarmManager wordt dikwijls gebruikt om persistent achtergrondwerk op te starten als de applicatie niet opgestart is.

WorkManager [8] De WorkManager is een krachtig en flexibel API voor het uitvoeren van taken die niet gekoppeld zijn aan een activity. Het is zowel een oplossing voor foreground services als voor task scheduling. Zo biedt ondersteuning voor zowel eenmalige langdurige

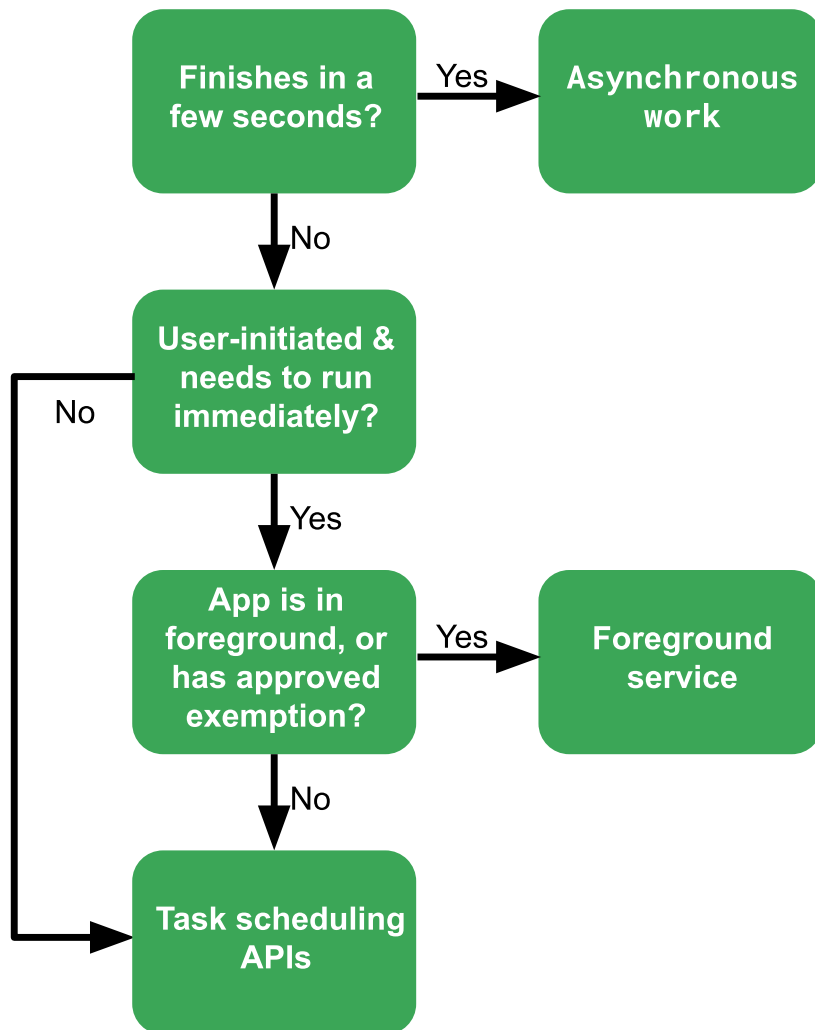


Figure 2.2: Keuze boom achtergrondwerk voor event driven tasks [6]

(one-time) als periodieke (periodic) taken. Voor achtergrondtaken die regelmatig moeten worden uitgevoerd, zoals het verzamelen van locatiegegevens, is de periodieke versie geschikt. Hierbij kan een taak bijvoorbeeld elke 15 minuten worden uitgevoerd, met een maximale duur van 10 minuten per taak. Voor taken die slechts één keer hoeven te worden uitgevoerd, al dan niet langdurig, kan de eenmalige versie worden gebruikt. Het is echter een belangrijk punt om op te merken dat langdurige taken in de foreground moeten lopen. Dit betekent dat er een permanente notificatie zichtbaar moet zijn, wat de gebruiker kan storen.

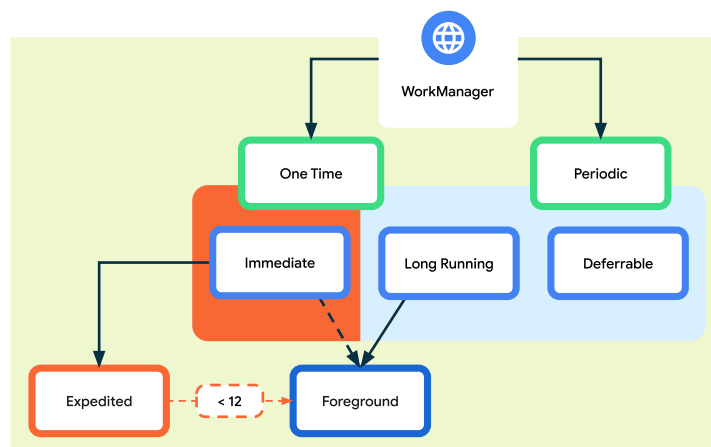


Figure 2.3: Android WorkManager

Coroutines [9] Coroutines vormen geen API voor het aanbieden van persistente services, maar zijn wel de aangewezen oplossing voor het uitvoeren van zware of blocking taken buiten de hoofdthread van een activity. Coroutines kunnen worden beschouwd als lichtgewicht alternatieven voor threads en bieden een efficiënte manier om asynchrone taken uit te voeren zonder de gebruikersinterface te blokkeren. Ze zijn echter gekoppeld aan de lifecycle van de aanroepende component (zoals een activity of fragment). Dit betekent dat wanneer de activity wordt gestopt, ook de bijbehorende coroutines worden beëindigd. Hierdoor zijn coroutines niet geschikt voor persistent achtergrondwerk dat moet blijven draaien, ongeacht de status van de activity. Dit zou dus een oplossing kunnen zijn om eenmalige een locatie op te vragen.

2.2 Locatie obfuscatie

In de studie van obfuscatie heb je 2 paden: hoe Android het aanpakt en voorgaand onderzoek losstaand van Android.

2.2.1 Android's oplossing

Voordat er dieper wordt ingegaan op locatie-obfuscatie in Android, is het belangrijk om de werking van de locatievoorzieningen in Android te bespreken. Android biedt twee primaire opties om locatiegegevens op te vragen: de standaard open-source implementatie en de Google Play Services-implementatie.

De standaard open-source implementatie leest enkel de data uit van de sensoren en voert geen verdere berekeningen of obfuscatie-algoritmes uit. Deze aanpak biedt ontwikkelaars meer vrijheid, maar brengt ook enkele nadelen met zich mee. Zo worden applicaties die deze versie gebruiken vaker gemarkeerd als potentieel kwaadaardig, omdat ze minder geïntegreerd zijn in het Google Android-ecosysteem.

De Google Play Services-implementatie, daarentegen, maakt gebruik van de Fused Location Provider. De Fused Location Provider combineert gegevens van verschillende sensoren, zoals Global Positioning System (GPS), wifi, mobiele netwerken en zelfs bewegingssensoren, om een zo nauwkeurig mogelijke locatie te berekenen. Dit proces wordt *sensor fusion* genoemd. De Fused Location Provider biedt ontwikkelaars een API om locatiegegevens op te vragen, waarbij automatisch de meest geschikte bron wordt gekozen op basis van de context, zoals batterijverbruik, nauwkeurigheid en beschikbaarheid van sensoren. Bovendien ondersteunt de Fused Location Provider zowel real-time locatie-updates als eenmalige locatieverzoeken, wat het een flexibele oplossing maakt voor uiteenlopende toepassingen.

In de Fused Location Provider zijn er verschillende parameters om locatiegegevens te verkrijgen. De meest nauwkeurige methode gebruikt alle sensoren inclusief GPS-satellieten, maar alternatieve methoden zoals enkel het gebruik van lokale telefoonmasten en wifi-netwerken is ook een mogelijkheid. Deze alternatieve methoden introduceren een inherente vorm van obfuscatie, aangezien de nauwkeurigheid doorgaans lager is dan die van GPS. Echter, in dichtbevolkte stedelijke gebieden kan dit juist leiden tot een hogere nauwkeurigheid dan GPS, omdat gebouwen GPS-signalen kunnen blokkeren.

Verder nog, sinds Android 12 hebben gebruikers de mogelijkheid gekregen om alleen toegang tot een benaderde locatie (*approximate location*) toe te staan, wat een extra dimensie toevoegt aan de discussie over privacy en nauwkeurigheid. De coarse location-functionaliteit in Android maakt geen gebruik van GPS-satellieten en voegt bovendien een extra laag van obfuscatie toe aan de locatiegegevens. Hoe deze obfuscatie precies wordt toegepast, is echter niet publiekelijk gedocumenteerd. Door dit zijn er preliminaire onderzoeken uitgevoerd waarover later meer.

2.2.2 Voorgaand onderzoek

Naast de oplossing van Android en nog voor Android 12 werd er al onderzoek gedaan naar de mogelijke gevaren en oplossingen.

Aanval van Kevin De Boeck [10] In eerdere onderzoeken, uitgevoerd door Kevin De Boeck, werd aangetoond dat locatie-obfuscatie een dringende noodzaak is. Uit data-analyse op een dataset van locatiegegevens in Frankrijk, verzameld over een periode van een week, bleek dat gevoelige locaties eenvoudig konden worden geïdentificeerd. Denk maar aan als er 50 punten op een heel afgelegen plek liggen dat dit misschien een overheids instelling kan zijn dat geheim moet blijven. Ook op vlak van het individu kon makkelijk een thuis of andere persoonlijke locaties gevonden worden. Verder konden er sociale grafen gemaakt worden en zelfs een volledige vakantie reis in kaart gebracht worden.

Oplossing Toon Dehaene [11] Voortbouwend op deze bevindingen heeft Toon Dehaene onderzoek gedaan naar methoden om GPS-locaties te obfusceren. Hiervoor werd een library ontwikkeld in Rust met behulp van PyO3, zodat deze als Python-module kon worden gebruikt voor het verwerken van grote datasets. Het doel was om locatie-traces op serverzijde te obfusceren. Naast de implementatie van parallelisatie was het kernalgoritme voor obfuscatie als volgt:

- Wanneer er onvoldoende tijd is verstreken sinds de vorige locatie, wordt simpelweg de vorige locatie opnieuw doorgestuurd.
- Vervolgens wordt het concept van privacycirkels toegepast: als de nieuwe locatie binnen een bestaande privacycirkel valt, wordt het middelpunt van deze cirkel samen met de huidige tijdstempel doorgestuurd.
- Valt de locatie buiten alle bestaande cirkels, dan wordt er een nieuwe cirkel aangemaakt met een willekeurige hoek en een straal die willekeurig gekozen wordt tussen 0 en de huidige maximale straal. De gebruiker valt dan altijd binnen deze nieuw gegenereerde cirkel, waarvan het middelpunt wordt doorgestuurd.

Dit algoritme introduceert dus ruis op de coördinaten en, belangrijker nog, voorkomt dat derden door het nemen van gemiddelden van geobfusceerde locaties alsnog gevoelige locaties kunnen achterhalen. Het algoritme vormde tevens de basis voor de standaardimplementatie van obfuscatie in de Android-library, wat verderop wordt toegelicht. Dit omdat dit algoritme uiterst geschikt is voor real-time data obfuscatie, waar als andere obfuscatie-methoden alle data op voorhand nodig hebben.

Testen van de oplossing In verder onderzoek van Toon Dehaene werd nagegaan of deze geobfusceerde locaties konden worden teruggevonden via gridsnapping. Dit werd onderzocht om de effectiviteit van locatieobfuscatie in afgelegen gebieden te evalueren. Wanneer er zich slechts één straat binnen de straal van een privacycirkel bevindt, is het immers visueel eenvoudig om de oorspronkelijke route te reconstrueren. (Dit roept de vraag op of de straal van de privacycirkel dynamisch zou moeten worden aangepast op basis van de bebouwingsdichtheid.)

Een bijkomende onderzoeksvraag was of het mogelijk is om een obfuscatiemethode te ontwikkelen waarbij de geobfusceerde locatie bewust op een willekeurige, maar plausibele locatie binnen een grid wordt geplaatst, zodat de geobfusceerde locaties in afgelegen gebieden realistischer lijken. Uit de resultaten bleek echter dat deze aanpak een averechts effect had. Verder onderzoek naar deze methode valt buiten de scope van deze thesis.

2.3 Preliminare experimenten

Aangezien de Android Developer-website niet alle vragen beantwoorde en er nog veel onduidelijkheden zijn, is een deel van de state-of-the-art gewijd aan onderzoek naar hoe deze variabelen het uiteindelijke resultaat beïnvloeden.

2.3.1 Verschillende nauwkeurighedsniveaus

De Fused Location Provider biedt vier prioriteitsniveaus voor het bepalen van de locatie, zoals beschreven in de Android-documentatie [12]:

- **Balanced power accuracy:** Gebruik deze instelling om locatieprecisie op te vragen binnen een stadsblok, wat een nauwkeurigheid van ongeveer 100 meter is. Dit wordt beschouwd als een grove nauwkeurigheid en verbruikt waarschijnlijk minder stroom. Met deze instelling maken de locatieservices waarschijnlijk gebruik van Wi-Fi en positionering van zendmasten. Houd er echter rekening mee dat de keuze van de locatieprovider afhankelijk is van veel andere factoren, zoals welke bronnen beschikbaar zijn.
- **High accuracy:** Gebruik deze instelling om de meest nauwkeurige locatie op te vragen. Met deze instelling is de kans groter dat de locatieservices GPS gebruiken om de locatie te bepalen.
- **Low power:** Gebruik deze instelling om precisie op stadsniveau aan te vragen, wat een nauwkeurigheid van ongeveer 10 kilometer is. Dit wordt beschouwd als een grove nauwkeurigheid en verbruikt waarschijnlijk minder stroom.
- **Passive:** Gebruik deze instelling als u een verwaarloosbare invloed op het stroomverbruik nodig heeft, maar locatie-updates wilt ontvangen wanneer deze beschikbaar zijn. Met deze instelling activeert uw app geen locatie-updates, maar ontvangt u locaties die door andere apps worden geactiveerd.

Naast de beschreven verschillen in nauwkeurigheid en energieverbruik, zijn er ook andere factoren die deze instellingen beïnvloeden. Uit experimenten bleek bijvoorbeeld dat de instelling *balanced power accuracy* wordt beïnvloed door de doze-modus of energiebesparingsmodus van Android. Tijdens tests werd vastgesteld dat een actief gebruikte telefoon een consistente locatie-trace genereerde, terwijl drie andere telefoons aanzienlijke hiaten vertoonden. Deze

hiaten werden veroorzaakt doordat de Fused Location Provider de callback niet activeerde vanwege de energiebesparende staten.

Tests met de prioriteiten *low power* en *passive* werden niet uitgevoerd. De *low power*-instelling was niet relevant voor onderzoek naar gebruikerslocatie-traces. Dit doordat de beloofde nauwkeurigheid 10km is. De *passive*-instelling niet omdat er te veel externe variabelen zijn, waardoor geen waardevolle resultaten konden worden verkregen. In voornaamste omdat de locatieupdates afhankelijk zijn van wat andere applicaties doen.

2.3.2 Coarse location Android

Om de coarse location-functionaliteit te onderzoeken, werd de applicatie tweemaal geïnstalleerd op dezelfde telefoon: één keer met coarse location-permissies en één keer met fine location-permissies. Beide applicaties waren geconfigureerd met dezelfde prioriteitsinstellingen om een eerlijke vergelijking mogelijk te maken. Een van de vragen was of de instelling *balanced power accuracy*, die geen gebruik maakt van satellieten, dezelfde resultaten zou opleveren als coarse location. Dit zou inzicht geven in de vraag of de obfuscatie uitsluitend voortkomt uit de inherente onnauwkeurigheid van de gebruikte technologieën, of dat Android actief een extra laag van obfuscatie toepast op de locatiegegevens. Als resultaat vinden we de volgende visualisatie Figuur 2.4 en Figuur 2.5. Hierbij is rood de trace met fine permissies en blauw coarse permissies.

- In beide voorbeelden zien we dat de coarse trace duidelijk minder punten bevat en verder van de werkelijke trace ligt dan de fine trace. Dit komt doordat, wanneer de nieuwe locatie te dicht bij de vorige ligt, de callback eenvoudigweg niet wordt opgeroepen.
- In het tweede voorbeeld zien we dat bij langdurig op dezelfde locatie blijven, de fine permissie relatief veel punten oplevert, waarbij de data varieert en zelfs een uitschieter vertoont. Dit is waarschijnlijk het gevolg van meer interferentie op de signalen in een betonnen gebouw. Opvallend is dat wanneer deze uitschieter optreedt, de coarse locatie eveneens een uitschieter in dezelfde richting vertoont. In dat geval wordt de callback dus wel opgeroepen.
- In het eerste voorbeeld zien we dat het coarse pad slechts vijf punten bevat. Ook ligt het pad meer dan de helft van de breedte van Gent verwijderd van de werkelijke trace. De obfuscatie-oplossing van Android kan niet beïnvloed worden door de gebruiker noch developer.
- In het tweede voorbeeld zien we linksboven op de Gentse Steenweg bij de Colruyt dat, wanneer er een volgend punt wordt berekend door de coarse-methode, dit punt ver genoeg van de vorige punten ligt. Dit kan mogelijk een extra privacymaatregel zijn.

Uit deze bemerkingen kunnen we concluderen dat bovenop de inherente onnauwkeurigheid ook nog extra obfuscatie gebeurt.

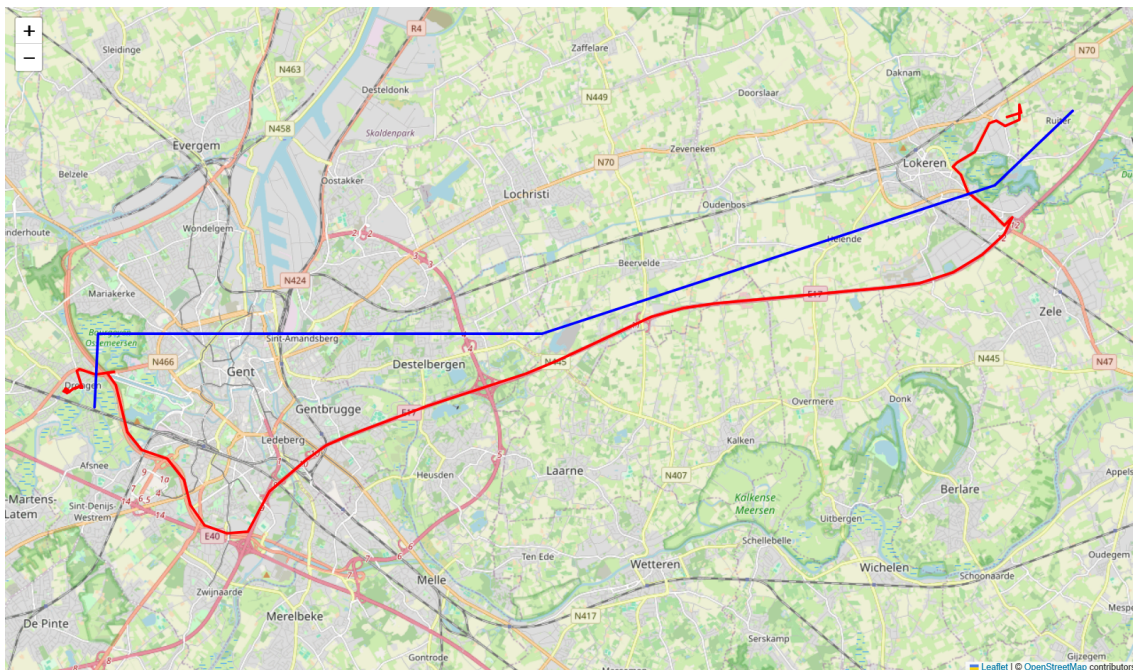


Figure 2.4: Vergelijking coarse permissies en fine permissies 1

Een interessante vraag die voortkwam uit deze experimenten, was hoe de obfuscatie van Android precies werkt. Om dit te onderzoeken, werden de coarse location-gegevens van eerdere tests naast elkaar gelegd. Zoals te zien is in Figuur 2.6, leveren de traces van op verschillende dagen maar wel hetzelfde apparaat steeds dezelfde coördinaten op. Dit wijst erop dat Android een soort privacycirkels implementeert, vergelijkbaar met eerder besproken algoritmes, om aanvallen te voorkomen waarbij gemiddelden over lange periodes worden gebruikt om gevoelige locaties te identificeren.

Een vervolgonderzoek richtte zich op de consistentie van dit algoritme. De vraag was of de geobfusceerde locaties consistent worden berekend op basis van nabijgelegen telefoonmasten, zodat dezelfde locaties op verschillende apparaten identiek zouden zijn. Uit de resultaten, weergegeven in Figuur 2.7, blijkt echter dat dit niet het geval is. Dit suggereert dat Android's obfuscatie-algoritme niet uitsluitend gebaseerd is op vaste berekeningen van telefoonmastlocaties, maar mogelijk ook andere factoren in overweging neemt.

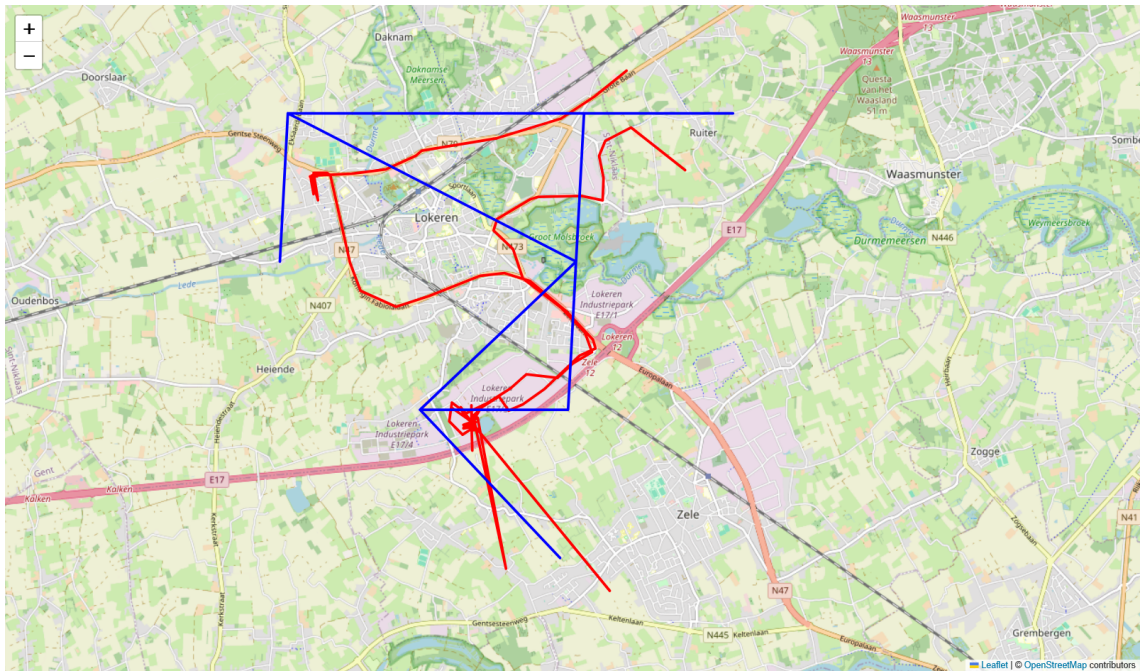


Figure 2.5: Vergelijking coarse permissies en fine permissies 2

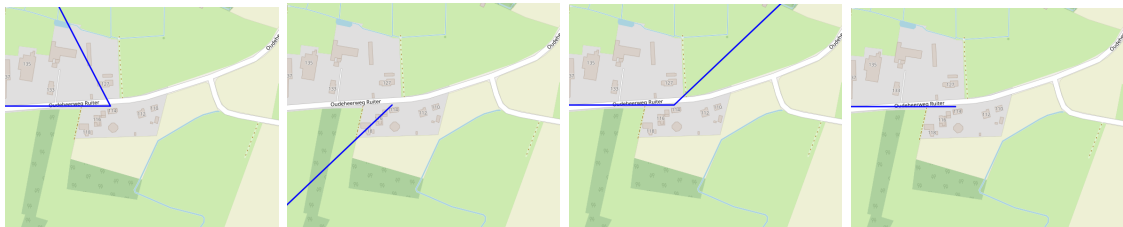


Figure 2.6: Vergelijking coarse permissie op één apparaat

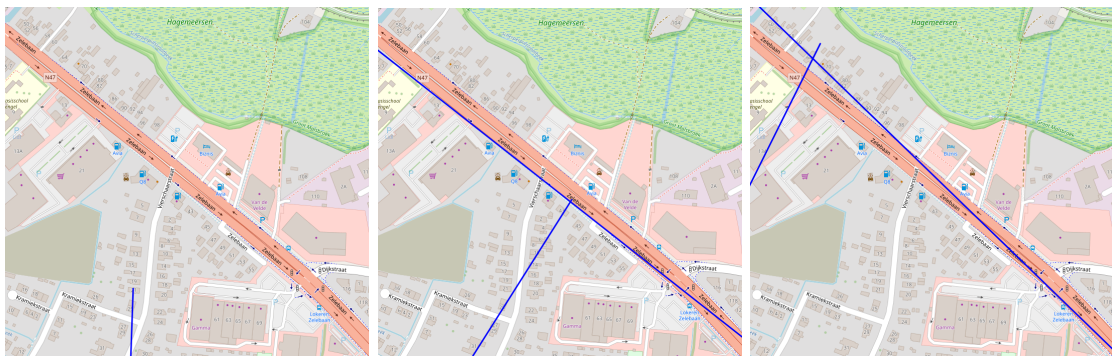


Figure 2.7: Vergelijking coarse permissie op meerdere apparaten

3

Design

Vooraleer de implementatie van de library kan starten, is een grondig ontwerp noodzakelijk. Dit ontwerp wordt in belangrijke mate bepaald door de beperkingen en mogelijkheden van het Android-platform. Daarnaast zijn de functionele vereisten en onderliggende veronderstellingen essentieel voor het uiteindelijke ontwerp.

3.1 Vereisten

De doelstellingen van dit onderzoek zijn als volgt geformuleerd.

Library Het ontwikkelen van een library met algemene bruikbaarheid: Het streven is om een library te creëren die in diverse applicaties kan worden toegepast. Deze library moet het proces van het aanvragen van permissies, initialisatie, gebruik en uitbreiding zo efficiënt mogelijk ondersteunen. Dit impliceert dat de library aan onderstaande vereisten moet voldoen om in uiteenlopende soorten applicaties te kunnen werken.

- **Flexibel:** De library mag niet beperkend zijn in vergelijking met de standaardinstellingen van de Android-implementatie en moet voldoende configuratiemogelijkheden bieden.
- **Modulair:** Gebruikers moeten in staat zijn om hun eigen obfuscatie-algoritmes te ontwikkelen en te integreren. De code dat ze niet horen te veranderen hoort louter een facade te zijn.
- **Eenvoudig te integreren:** De library moet functioneren met minimale setup en gebruiksvriendelijke integratiemogelijkheden bieden. Met minimale setup wordt bedoeld

dat de library volledig moet functioneren met zo weinig mogelijk lines of code.

- **Lage leercurve:** Een van de aannames is dat de ontwikkelaar mogelijk weinig tot geen kennis heeft van locatie services of background work in Android. Daarom moeten de standaardinstellingen een solide basis bieden voor het veilig omgaan met locatiegegevens.

Privacy Op het vlak van privacy geldt als vereiste dat alle gevoelige data het toestel niet verlaat en dus ook dat de data niet toegankelijk is voor andere applicaties.

Ook moet de geobfusceerde locatie voldoende accuraat blijven voor het beoogde gebruik, zonder dat het mogelijk is een individu te identificeren of te traceren. Voor deze obfuscatietechniek wordt voortgebouwd op het werk van Toon Dehaene, zoals eerder besproken. Dit algoritme zal moeten aangepast worden om geschikt te zijn voor verwerking van datastromen in plaats van parallelle verwerking, en zal geïmplementeerd moeten worden in Kotlin.

3.2 Veronderstellingen

Om de werking van de library te kunnen garanderen, worden de volgende veronderstellingen gemaakt:

- **Goede intenties van de ontwikkelaar:** Er wordt aangenomen dat de ontwikkelaar te goeder trouw handelt. De library kan immers aangepast, omzeild of gewoon weg niet gebruikt worden, waardoor het alsnog mogelijk is om de echte locatie door te sturen.
- **Integriteit van het Android-systeem:** Er wordt verondersteld dat het Android-systeem waarop de applicatie draait niet gecompromitteerd is. Dit betekent dat er geen derde partij of de eigenaar roottoegang heeft, aangezien dit zou toelaten om privacygevoelige gegevens rechtstreeks uit te lezen.
- **Correct gebruik van onderzoeks- en debugmodus:** De applicatie zal een onderzoeks- en debugmodus bevatten. Het is essentieel dat deze modus niet in productieomgevingen wordt gebruikt, omdat dit zou kunnen leiden tot het versturen van de echte locatie samen met de geobfusceerde locatie.
- **Normaal gebruik door de gebruiker:** De permanente opslag van gegevens wordt gekoppeld aan de installatie van de applicatie. Bij frequente herinstallatie van de applicatie worden voorgaande privacycirkels verwijderd. Dit zou het mogelijk maken om een gemiddelde te nemen van deze nieuwe privacycirkels, waardoor alsnog gevoelige locaties achterhaald kunnen worden.

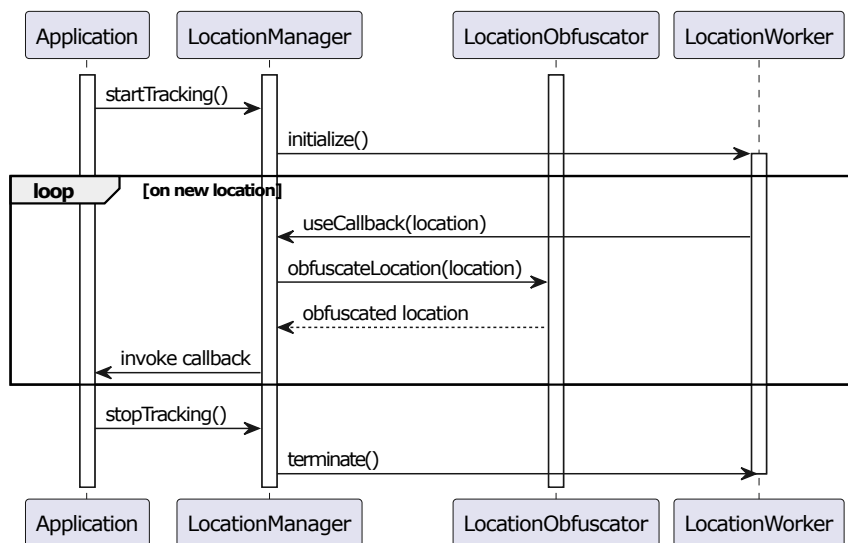


Figure 3.1: Flow diagram van continue locatieverzameling

3.3 Ontwerp

Het ontwerp van de library is gebaseerd op het principe van een facade, waarbij de library fungeert als een vereenvoudigde interface voor de standaard Android-functionaliteiten. Dit betekent dat alle variabelen en functionaliteiten aanwezig moeten zijn alsof de ontwikkelaar rechtstreeks met de standaard Android-implementatie werkt, maar met de toegevoegde voordelen van de library.

Met bovenstaande ontwerpprincipes in gedachten illustreert Figuur 3.1 hoe één enkele functie het verzamelen van locatiegegevens kan starten. Figuur 3.1 toont vier actoren. De applicatie zelf valt buiten de controle van de library en is dus niet te beïnvloeden. De `LocationManager` fungeert als het centrale aanspreekpunt dat de volledige werking coördineert, zodat de developer ook maar met één aanspreekpunt bezorgd moet zijn. De `LocationObfuscator` vormt het modulaire onderdeel dat door de ontwikkelaar naar wens kan worden aangepast of vervangen. Tot slot is er de `LocationWorker`, die als aparte klasse wordt geïmplementeerd omdat deze de `Worker` van de `WorkManager-API` van Android moet uitbreiden. Deze scheiding zorgt voor een duidelijke structuur en maakt het systeem flexibel en uitbreidbaar. Het is echter belangrijk om op te merken dat deze implementatie de functie `RequestPermission` niet omvat, aangezien deze losstaand moet opgeroepen geweest zijn. Want in de flow diagram staan geen Preliminaire checks, als de applicatie geen toegang heeft tot locaties zal deze niet proberen locaties op te vragen.

Verder is de `LocationManager` een Singleton [13] gemaakt. Singletons zorgen ervoor dat er slechts één instantie van een klasse bestaat binnen de applicatie. Zodat wanneer de developer twee-maal de `LocationManager` probeert te initialiseren de applicatie niet eventueel crashed doordat beide instanties locaties opvragen en het limiet overschrijden.

Daarnaast moeten alle standaardinstellingen en functionaliteiten van de library een solide basis

bieden, zodat ontwikkelaars zonder uitgebreide kennis van Android-locatieservices de library direct kunnen gebruiken. Dit omvat het bieden van een goede standaardimplementatie die flexibel genoeg is om aanpasbaar te zijn voor meer geavanceerde toepassingen. Ook moeten de setters rekening houden met de grenzen van het Android systeem. Bijvoorbeeld als de applicatie enkel coarse permissies heeft mag de app niet frequenter dan één keer om de 2 minuten een locatie vragen of de volledige FusedLocationProvider zal niet meer werken.

4

Implementatie

De eerste stap in deze thesis was het onderzoeken van hoe Android omgaat met locatiegegevens en het implementeren van deze functionaliteit in een testapplicatie. Zodra deze applicatie functioneel was, werden de eerste preliminaire testen uitgevoerd en werd de nuttige code afgesplitst om de basis te vormen van de Android-library.

4.1 Overzicht library

De structuur van de library is weergegeven in Figuur 4.1. Het merendeel van de functionaliteit is ondergebracht in de klasse `LocationManager`, die fungeert als het centrale beheerpunt voor locatieverzameling. De klasse `LocationWorker` vormt de implementatie die vereist is om de `WorkManager` te benutten voor achtergrondtaken. Daarnaast biedt de interface `LocationObfuscator` een abstractielaag waarmee ontwikkelaars eigen obfuscatie-algoritmes kunnen implementeren.

4.1.1 Locatie tracking

Het centrale onderdeel van de library, namelijk de locatieverzameling, wordt beheerd door de `LocationManager`, die fungeert als het centrale aanspreekpunt. Deze klasse beheert alle benodigde variabelen voor locatieverzameling en biedt setters met ingebouwde validaties om de stabiliteit te waarborgen. Variabelen die specifiek zijn voor obfuscatie worden echter beheerd door de obfuscator zelf, aangezien deze buiten de verantwoordelijkheid van de `LocationManager` vallen.

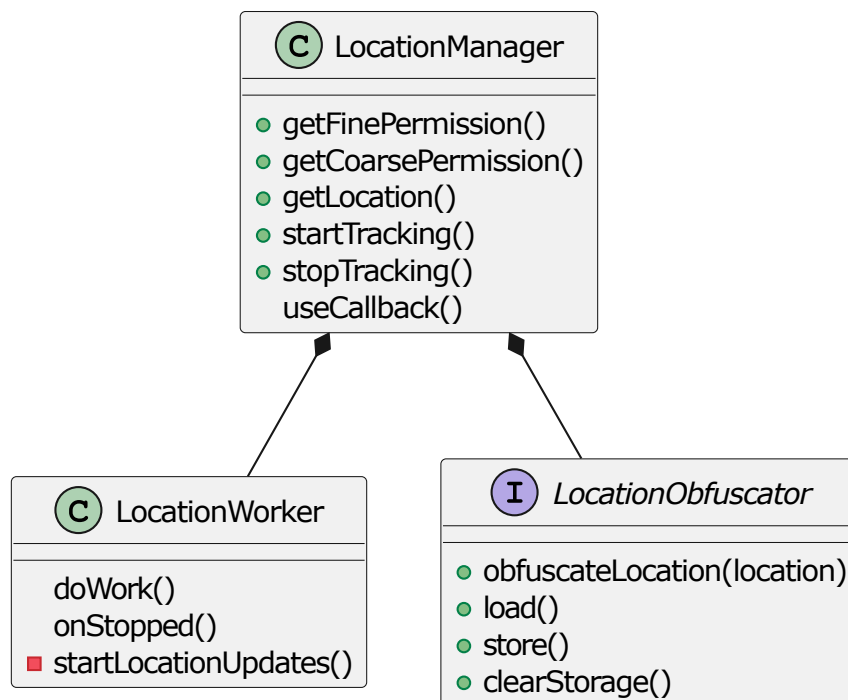


Figure 4.1: Klasse diagram van de library

4.1.1.1 Background en foreground services

Voordat locaties verzameld kunnen worden, moeten enkele beperkingen in acht worden genomen. Allereerst mogen locaties niet worden opgevraagd in dezelfde thread als de activity waarmee de gebruiker interacteert. Zelfs als dit in een aparte thread vanuit de hoofdactivity wordt opgestart. Is dit geen oplossing voor permanente locatieverzameling want Android zal deze beëindigen vanwege de strikte beperkingen op services wanneer de activity wordt gestopt of vernietigd.

Om taken uit te voeren wanneer de hoofdactivity niet actief is, kan gebruik worden gemaakt van foreground services of background tasks. Voor deze library is gekozen voor de WorkManager, vanwege de flexibiliteit en betrouwbaarheid die deze biedt. De WorkManager kan zowel functioneren als een foreground service als een periodieke background task, waardoor de zelfde klasse die de Worker van WorkManager implementeert kan gebruikt worden voor beide.

Foreground services Een foreground service wordt gebruikt om langdurige taken uit te voeren die een hogere prioriteit vereisen. Om een service als foreground service te configureren, moet in de hoofdactiviteit een notificatiekanaal worden aangemaakt. Dit notificatiekanaal zorgt ervoor dat de gebruiker visueel geïnformeerd wordt over de achtergrondactiviteit. Nadat het notificatiekanaal is aangemaakt, kan de Worker, een klasse die de vereiste interface van de WorkManager implementeert, een notificatie genereren en weergeven. Dit gebeurt als

volgt in de LocationManager, zoals weergegeven in onderstaande code, die een referentie heeft naar de hoofdactiviteit. De LocationManager maakt een notificatiekanaal aan met IMPORTANCE_LOW om de gebruiker zo min mogelijk te storen.

```
private fun setupNotificationChannel() {
    val channelId = "locationTrackingChannel"
    val channelName = "Location Tracking"
    val channelDescription = "Location tracking is active"
    val notificationManager =
        activity.getSystemService(NOTIFICATION_SERVICE) as
        ↳ NotificationManager

    // Create notification channel if not already created
    val channel =
        NotificationChannel(channelId, channelName,
            ↳ NotificationManager.IMPORTANCE_LOW).apply {
                description = channelDescription
            }
    notificationManager.createNotificationChannel(channel)
}
```

Dan kan de LocationWorker ingrijpen op dit notificatiekanaal door dezelfde ID aan te spreken. De nodige info is een notificatie voor de foregroundInfo, zodat Android weet waarvoor de notificatie bedoeld is. De implementatie Hiervoor is te zien in onderstaande code.

```
val notification =
    NotificationCompat.Builder(applicationContext,
        ↳ "locationTrackingChannel")
        .setContentTitle("Location Tracking")
        .setTicker("Location Tracking")
        .setContentText("Tracking your location in the background")
        .setSmallIcon(R.drawable.ic_launcher_background)
        .setOngoing(true)
        .build()

val foregroundInfo =
    ForegroundInfo(1001, notification,
        ↳ ServiceInfo.FOREGROUND_SERVICE_TYPE_LOCATION)
setForegroundAsync(foregroundInfo)
```

Zodra de notificatie is geactiveerd, kan de Worker ononderbroken blijven werken totdat deze zichzelf beëindigt of totdat de functie cancelUniqueWork van de WorkManager wordt

aangeropen vanuit de hoofdactivity. Deze aanpak zorgt ervoor dat de taken betrouwbaar en efficiënt worden uitgevoerd, zelfs wanneer de applicatie zich niet op de voorgrond bevindt.

Background tasks Wanneer een visuele indicatie van achtergrondactiviteit niet vereist of mogelijk is, kan gebruik worden gemaakt van periodieke taken. Historisch gezien werd hiervoor de `AlarmManager` gebruikt, die functioneerde als een soort cron job in Linux. Echter, vanwege strengere beperkingen op maximale duur en frequentie, is deze aanpak minder geschikt geworden.

De `WorkManager` biedt een modern en aanbevolen alternatief voor het uitvoeren van periodieke taken. Er zijn echter enkele beperkingen: taken kunnen maximaal één keer per 15 minuten worden uitgevoerd en mogen niet langer dan 10 minuten actief zijn. Ondanks deze restricties is de `WorkManager` robuust, energiezuinig en goed geïntegreerd met het Android-platform, waardoor achtergrondprocessen betrouwbaar beheerd kunnen worden. Qua implementatie wordt dezelfde codebasis gebruikt als voor de foreground service, maar zonder het opzetten van het notificatiekanaal zoals hierboven beschreven.

Integratie in de library Doordat de `WorkManager` beide vereisten kan afhandelen, is er slechts één klasse nodig die zowel foreground services als periodieke taken implementeert. Deze klasse, die de `Worker`-interface implementeert, bevat tevens de `LocationCallback` die vereist is voor de `FusedLocationProvider`. Deze callback speelt alle resultaten direct door naar de `LocationManager`, die deze resultaten vervolgens verwerkt volgens de ingestelde configuratie. Deze aanpak minimaliseert de complexiteit en verhoogt de herbruikbaarheid van de code.

Een bijkomende reden om de `LocationManager` als een Singleton te implementeren, is de beperking van de `LocationWorker` om complexe structuren of referenties door te geven. Zo is de initialisatie van een `Worker` beperkt tot primitieve datatypes. Hierdoor kan er geen referentie doorgegeven worden naar een stop signaal. Want door gebruik te maken van een Singleton-patroon kan de `LocationWorker` de `LocationManager` initialiseren en een referentie behouden naar diens variabelen. Zo bevat de `LocationManager` bijvoorbeeld een boolean genaamd `running`, waarmee kan worden aangegeven of de locatieverzameling actief is. Indien de locatieverzameling wordt stopgezet, kan de `LocationWorker` deze status uitlezen en zichzelf correct beëindigen.

4.1.1.2 Permissies

Een voordeel van Android is sandboxing, waardoor alles waar de applicatie probeert toegang tot te krijgen gemonitord kan worden. Hierdoor is het noodzakelijk dat de applicatie expliciet toestemming vraagt voor bepaalde functionaliteiten; indien deze toestemming niet wordt verkregen, zal de applicatie bij een toegangsverzoek crashen. Alle benodigde permissies horen ook in het manifestbestand vermeld te worden. Dit manifestbestand is vergelijkbaar aan een

package.json voor een nodejs project. Hierin staan alle libraries dat de applicatie nodig heeft, maar dus ook de permissies die de app eventueel kan opvragen en de services dat de app eventueel gebruik van kan maken. Dit is nodig om de applicatie te laten compileren naar een APK.

Android onderscheidt hierbij twee categorieën permissies. De eerste categorie omvat permissies die uitsluitend in het manifestbestand van de applicatie moeten worden opgenomen. Voor deze permissies is geen expliciete toestemming van de gebruiker vereist tijdens runtime. De tweede categorie omvat permissies die niet alleen in het manifestbestand moeten worden opgenomen, maar ook expliciet tijdens runtime door de gebruiker moeten worden goedgekeurd. Dit gebeurt via de functie `RequestPermission`, waarbij Android een dialoogvenster toont om de gebruiker om toestemming te vragen. Het is belangrijk dat de bijbehorende activiteit correct is geïnitieerd voordat deze aanvraag wordt gestart; anders beschouwt Android dit als een onveilige actie en zal de activiteit worden beëindigd.

Voor deze applicatie zijn de permissies als volgt:

- **Enkel manifest:** `ACCESS_BACKGROUND_LOCATION`, `FOREGROUND_SERVICE` en `FOREGROUND_SERVICE_LOCATION`
- **Met `RequestPermission`:** `ACCESS_COARSE_LOCATION` en `ACCESS_FINE_LOCATION`

Ook het afhandelen van deze permissies is geïmplementeerd in de library in de functies `getFinePermission` en `getCoarsePermission`. De werking van deze functies verloopt als volgt:

- Eerst wordt gecontroleerd of de applicatie de betreffende permissie al heeft. Indien dit het geval is, retourneert de functie `true`.
- De functie kan ook enkel als controle worden gebruikt, afhankelijk van de parameter `askIfNecessary`.
- Indien het aanvragen van de permissie is toegestaan, wordt nagegaan of de gebruiker de permissie eerder heeft geweigerd. In dat geval wordt de gebruiker doorgestuurd naar de instellingen, aangezien de standaard pop-up dan niet meer wordt getoond. Is het de eerste aanvraag, dan verschijnt de reguliere pop-up.
- Tot slot wordt de interne variabele, die als `LiveData` wordt bijgehouden, aangepast en wordt het resultaat geretourneerd.

```
private val askCoarse = activity.registerForActivityResult(  
    ActivityResultContracts.RequestPermission()  
) { res ->  
    this._coarsePermission.value = res  
}
```

```

fun getCoarsePermission(askIfNecessary: Boolean = false): Boolean {
    this._coarsePermission.value = activity.checkSelfPermission(
        Manifest.permission.ACCESS_COARSE_LOCATION
    ) == PackageManager.PERMISSION_GRANTED
    if (this._coarsePermission.value == true) {
        this.logDebug("already granted", LogLevel.VERBOSE)
        return true
    }

    if (askIfNecessary) {
        if (ActivityCompat.shouldShowRequestPermissionRationale(
            activity,
            Manifest.permission.ACCESS_COARSE_LOCATION
        )) {
            val intent =
                ↪ Intent(android.provider.Settings.ACTION_APPLICATION_DETAILS_SETTINGS)
            intent.data = Uri.parse("package:${activity.packageName}")
            startActivity(activity, intent, null)
        } else
            ↪ askCoarse.launch(Manifest.permission.ACCESS_COARSE_LOCATION)
    }
    return this._coarsePermission.value == true
}

```

4.1.2 Obfuscatie

Wanneer de Worker de callback van de LocationManager aanroept met het locatie-resultaat van de FusedLocationProvider, zijn er twee mogelijke paden:

- **Zonder obfuscator:** Indien er geen locatieobfuscator is ingesteld, fungeert de library als een eenvoudige facade. Als er een callbackfunctie is opgegeven, worden de coördinaten en het tijdstip als een Triple doorgegeven aan deze callback.
- **Met obfuscator:** Indien er wel een locatieobfuscator is ingesteld, wordt het resultaat van de FusedLocationProvider eerst naar de obfuscator gestuurd. Deze retourneert een geobfusceerde Triple met coördinaten en tijdstip, of null indien geen resultaat beschikbaar is. Indien een debug-callback is ingesteld, ontvangt deze zowel de originele als de geobfusceerde locatie. Alleen als er een geldig resultaat én een callbackfunctie is, wordt de callback aangeroepen met de geobfusceerde locatie.

```

fun useCallback(location: Location) {

```

```

if (_locationObfuscator == null) {
    _callback?.invoke(Triple(location.latitude, location.longitude,
        ↪ location.time))
} else {
    val obfuscatedLocation: LatLonTs? =
        this._locationObfuscator!!.obfuscateLocation(location)
    _debugCallback?.invoke(
        Triple(
            location.latitude,
            location.longitude,
            location.time
        ), obfuscatedLocation
    )
    // no callback if no result
    obfuscatedLocation?.let { x -> _callback?.invoke(x) }
}
}

```

4.1.2.1 Standaard obfuscatie-algoritme

De library vereist een standaardimplementatie voor een obfuscatieklasse. Hiervoor wordt verwezen naar het werk van Toon Dehaene, waarin het concept van privacyzones centraal staat. Het toevoegen van enkel ruis aan locatiegegevens kan immers leiden tot het risico dat een aanvaller door middel van gemiddelden alsnog gevoelige informatie kan achterhalen. Het algoritme, zoals weergegeven in Figuur 4.2, illustreert de werking van deze aanpak.

- Het proces start met het filteren van de bestaande privacycirkels om te bepalen of de nieuwe locatie binnen een van deze cirkels valt. Tijdens dit proces wordt tevens de privacycirkel bijgehouden waarvan het middelpunt het dichtst bij de huidige locatie ligt.
- Indien de huidige locatie niet binnen een van de bestaande privacycirkels valt, wordt er een nieuwe cirkel aangemaakt. Het middelpunt van deze cirkel wordt bepaald door vanuit de huidige locatie, een willekeurige hoek en een willekeurige afstand (tussen 0 en de vooraf ingestelde maximale straal) te kiezen. Deze maximale straal is configureerbaar.
- Vervolgens wordt gecontroleerd of de vorige locatie voldoende lang geleden is verzonden. Indien dit niet het geval is, retourneert de obfuscator een null-waarde.
- Om jitter te voorkomen, wordt nagegaan of de huidige locatie zich in meerdere privacycirkels bevindt én binnen de laatst gebruikte privacycirkel valt. In dat geval wordt deze laatst gebruikte cirkel opnieuw geselecteerd, ongeacht of een andere cirkel dichterbij ligt. In alle andere gevallen wordt het middelpunt van de dichtstbijzijnde privacycirkel samen met de huidige tijdstempel als resultaat gebruikt.

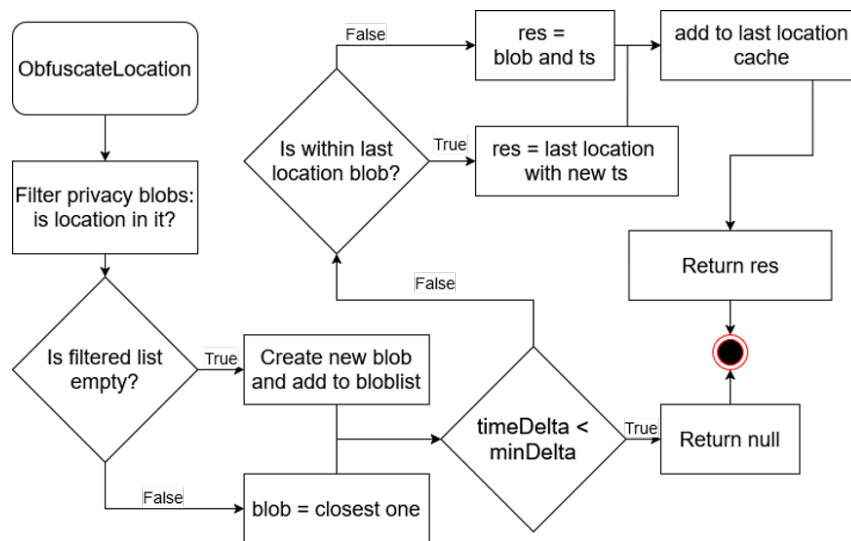


Figure 4.2: Flow diagram van het obfuscatie algoritme

De veranderingen tegenover het originele algoritme is dat er eerst gecheckt wordt of de gebruiker in een privacycirkel bevindt als voor het controleert of het lang genoeg geleden is. Daarnaast is een belangrijke toevoeging het mechanisme om jitter te voorkomen. Jitter kan optreden wanneer een gebruiker zich in het overlappende gebied van twee privacycirkels bevindt en de werkelijke locatie door kleine variaties (ruis) telkens net iets dichterbij een andere cirkel ligt. Zonder extra maatregelen zou het algoritme hierdoor continu kunnen wisselen tussen de twee cirkels, waardoor de uiteindelijke geobfusceerde locatie steeds verspringt. Dit maakt het voor een aanvaller mogelijk om, door het analyseren van deze verspringende locaties, de werkelijke positie van de gebruiker te verfijnen tot het overlappende gebied van de cirkels. Door altijd de laatst gebruikte privacycirkel te gebruiken zolang de gebruiker zich daarin bevindt, wordt dit risico geminimaliseerd.

4.1.3 Opslag

Omdat veel obfuscatie-algoritmes permanente opslag nodig hebben moet iedere obfuscatieklasse ook een load en store functie implementeren. Dit heeft als argument de `filesDir` van de hoofdactiviteit. Dit is een object waarmee de developer de opslag van de applicatie kan beïnvloeden. In de standaardimplementatie wordt dit gebruikt om voorgaande privacy cirkels op te slaan in een voor andere applicaties afgesloten file. [14]

4.1.4 Netwerk

De library heeft geen functionaliteit voor de locatie door te sturen naar een API endpoint. Dit moet geïmplementeerd worden in de callback function die de developer meegeeft aan de `LocationManager`. In de testapplicatie wordt de library, aanbevolen door Android zelf, `okhttp`

gebruikt om de locatie door te sturen.

4.2 Applicatie

De testapplicatie is ontworpen om de werking van de library te demonstreren en te testen, zoals geïllustreerd in Figuur 4.3. De gebruikersinterface bestaat uit verschillende instellingen en knoppen die direct gekoppeld zijn aan de functionaliteit van de `LocationManager` en de locatieobfuscator.

De eerste vier instellingen betreffen de configuratie van de locatieverzameling:

- **Priority:** Bepaalt de nauwkeurigheid en het energieverbruik van de `FusedLocationProvider`.
- **Foreground:** Geeft aan of de locatieverzameling als foreground service (met notificatie) of als background task wordt uitgevoerd.
- **Interval (ms):** Stelt het tijdsinterval in tussen opeenvolgende locatieverzoeken aan de `FusedLocationProvider`. Dit interval heeft geen invloed op het interval van de standaard locatieobfuscator.
- **Max runtime (min):** Bepaalt de maximale duur van de background service. Volgens de beperkingen van Android kan deze maximaal 10 minuten per 15 minuten actief zijn.

Daarnaast zijn er twee instellingen specifiek voor de standaardimplementatie van de locatieobfuscator:

- **Blob radius:** De maximale straal van een privacycirkel.
- **Time between privacy:** De minimale tijd tussen het versturen van opeenvolgende locaties.

Onder de instellingen bevinden zich zes knoppen met de volgende functies:

- **Permissies aanvragen:** De eerste twee knoppen vragen de benodigde permissies op bij de gebruiker. De status van deze permissies wordt weergegeven en geüpdatet via LiveData zoals te zien in de tweede screenshot.
- **Clear obfuscator storage:** Hiermee wordt de permanente opslag van de obfuscator gewist, zodat testen telkens met een schone lei kunnen starten.
- **Nieuwe UUID genereren:** Hiermee wordt een unieke identificatie gegenereerd voor elke test, zodat resultaten eenvoudig te onderscheiden zijn.
- **Tracking starten/stoppen:** De laatste twee knoppen zijn voor locatieverzameling. De status van de tracking wordt eveneens via LiveData weergegeven in de linker knop, zodat de interface altijd de actuele status toont.

In het onderste tekstvak worden de resultaten gelogd die via de callback (niet de debug-callback) worden doorgegeven. Dit biedt tijdens het testen een directe indicatie dat de applicatie actief locaties verwerkt.

De drie screenshots tonen respectievelijk:

- De initiële opstart van de applicatie zonder verleende permissies.
- De situatie waarin de permissieknoppen de status van verleende permissies weergeven.
- De applicatie tijdens actieve tracking, waarbij een locatie is gelogd in het tekstvak.

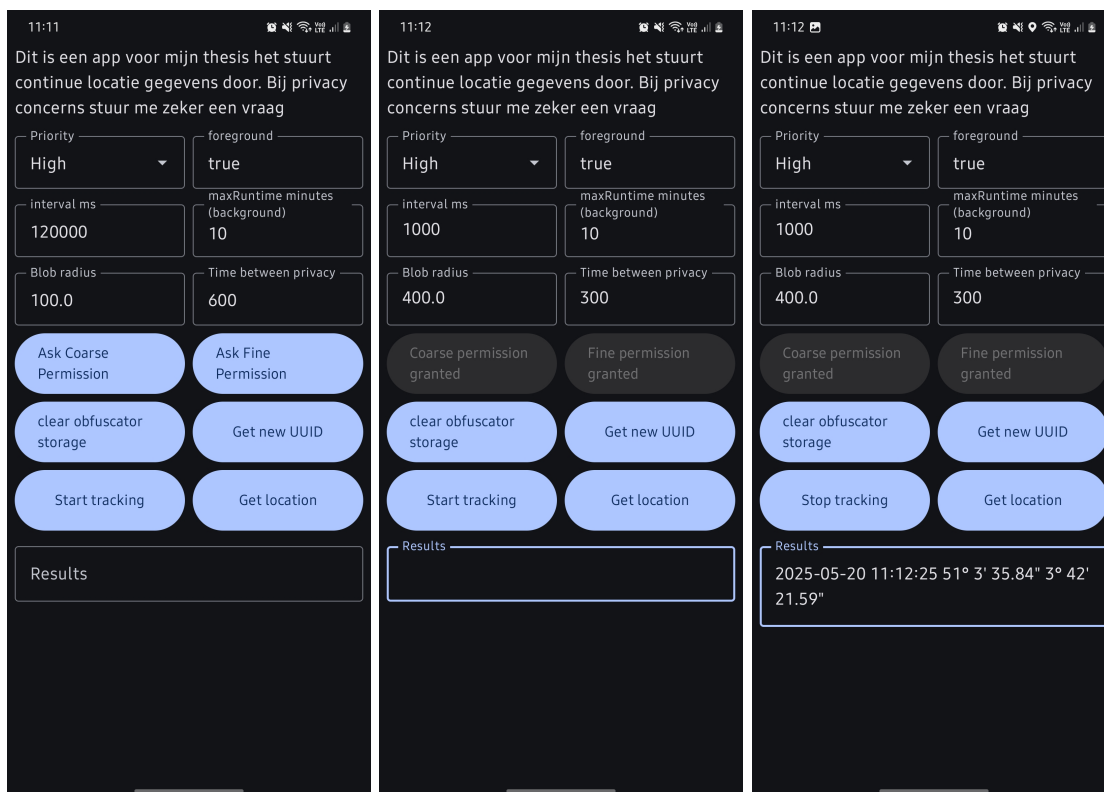


Figure 4.3: Visualisatie van de test applicatie

4.3 Dataverwerking

4.3.1 Verzameling

Voor de verzameling van data werd gebruik gemaakt van een Azure-webserver, specifiek opgezet voor studenten, met twee endpoints: `location` en `blobs`.

De `location`-endpoint ontving zowel de geobfusceerde als de originele locatiegegevens, afkomstig van de debug-callback, samen met relevante configuratieparameters. Deze gegevens werden opgeslagen in een CSV-bestand voor latere analyse.

De blobs-endpoint werd gebruikt om de privacycirkels door te sturen.

Om de afhankelijkheid van een continue internetverbinding te minimaliseren, werd een mechanisme geïmplementeerd waarbij de data lokaal werd opgeslagen wanneer er geen netwerkverbinding beschikbaar was. Zodra de verbinding werd hersteld, werden de opgeslagen gegevens alsnog doorgestuurd naar de server.

4.3.2 Verwerking

Voor de verwerking van de resultaten werden jupyter notebooks gehanteerd samen met de library Folium [15] om de resultaten visueel te kunnen representeren. In elke notebook is er een functie die de data inleest en locatiepunten die in de buurt van mijn privélocaties/privésfeer zijn weg filtert. Dit zorgt ervoor dat alle gevisualiseerde data gepubliceerd kan worden. De eerste visualisatie was het verschil tussen coarse en fine Permissies vergelijken voor een experiment. Dit was beide op balanced power accuracy gebeurd zodat beide geen GPS satellieten gebruikt. Het resultaat is te zien in het hoofdstuk experimenten in state of the art 2.42.5. Een tweede test betrof het gebruik van twee telefoons met coarse permissies en twee met fine permissies. Voor elk paar werd één telefoon ingesteld op high accuracy en de andere op balanced power accuracy. Deze test werd echter niet opgenomen in het preliminaire experimentenonderdeel van het hoofdstuk *State of the Art*, omdat er te veel externe factoren waren die de resultaten beïnvloedden, zoals bv. de beperkingen opgelegd door de energiebesparingsfunctionaliteiten van Android.

De overige tests richtten zich op het evalueren van de standaardimplementatie van het obfuscatie-algoritme in de library. Hierbij werden verschillende variabelen gevarieerd om de werking van het algoritme te analyseren en mogelijke verbeterpunten te identificeren. De inzichten die uit deze tests voortkwamen, werden gebruikt om de library verder te debuggen en te optimaliseren. Een uitgebreide bespreking van deze resultaten is te vinden in het hoofdstuk *Experimenten en resultaten*.

4.4 Conclusie

In dit hoofdstuk werd de implementatie van de Android-library uitgebreid besproken. De library is ontworpen om locatiegegevens op een privacyvriendelijke manier te verzamelen en te verwerken. Het centrale beheer van locatieverzameling wordt uitgevoerd door de `LocationManager`, terwijl de `WorkManager` zorgt voor een flexibele en betrouwbare uitvoering van zowel foreground services als periodieke background tasks.

De library integreert een standaard obfuscatie-algoritme gebaseerd op privacycirkels, waarmee gevoelige informatie effectief wordt beschermd. Ontwikkelaars hebben de mogelijkheid om aangepaste obfuscatieklassen te implementeren en gebruik te maken van debug-callbacks voor analyse. Permanente opslag wordt ondersteund om privacycirkels en andere gegevens veilig te bewaren.

Daarnaast werd een testapplicatie ontwikkeld om data te verzamelen via een Azure-webserver. Hierbij werden mechanismen geïmplementeerd om gegevens lokaal op te slaan bij netwerkuitval, wat de betrouwbaarheid van de oplossing verhoogt. Deze aanpak minimaliseert de afhankelijkheid van een continue internetverbinding.

5

Experimenten en resultaten

Naast de resultaten van de testen in verband met de werking van Android besproken in de state of the art. Gaat dit hoofdstuk over de uiteindelijke resultaten waarover de thesis gaat.

5.1 Visuele representatie

Voor het testen van de uiteindelijke library hebben we een specifieke route uitgestippeld die verschillende situaties omvat om de obfuscatie op diverse manieren te evalueren. De gevolgde route startte aan de Technologicampus Gent, waarna er naar de Korenmarkt werd gefietst. Daar werd gedurende twee uur op een terras gezeten om langdurig verblijf op één locatie te simuleren. Vervolgens werd er met de fiets naar de bibliotheek De Krook gereden, waar een korte stop werd ingelast om een kortstondig verblijf te testen. Daarna werd via een omweg naar station Dampoort gefietst, om vervolgens met de trein naar Lokeren te reizen en zo snel vervoer in een afgesloten ruimte te evalueren. Het resultaat van zo een trace is te zien in figuur 5.1.

In deze trace vallen enkele zaken op. Op locaties waar langere stops werden gemaakt, zoals op de campus, de Korenmarkt, De Krook en station Dampoort, ontstaat een cirkelvormig patroon door kleine variaties in de gemeten locatie. Bij het station is zelfs een uitschieter zichtbaar, vermoedelijk veroorzaakt door beweging onder het spoor waar het GPS-sigitaal minder duidelijk is.

Verder is het opvallend dat het gefietste traject zeer accuraat wordt weergegeven, terwijl het deel van de route per trein sterk afwijkt van het werkelijke traject. Dit kan te wijten zijn aan het kooi-van-Faraday-effect van de trein of aan de hogere snelheid waarmee wordt gereisd.

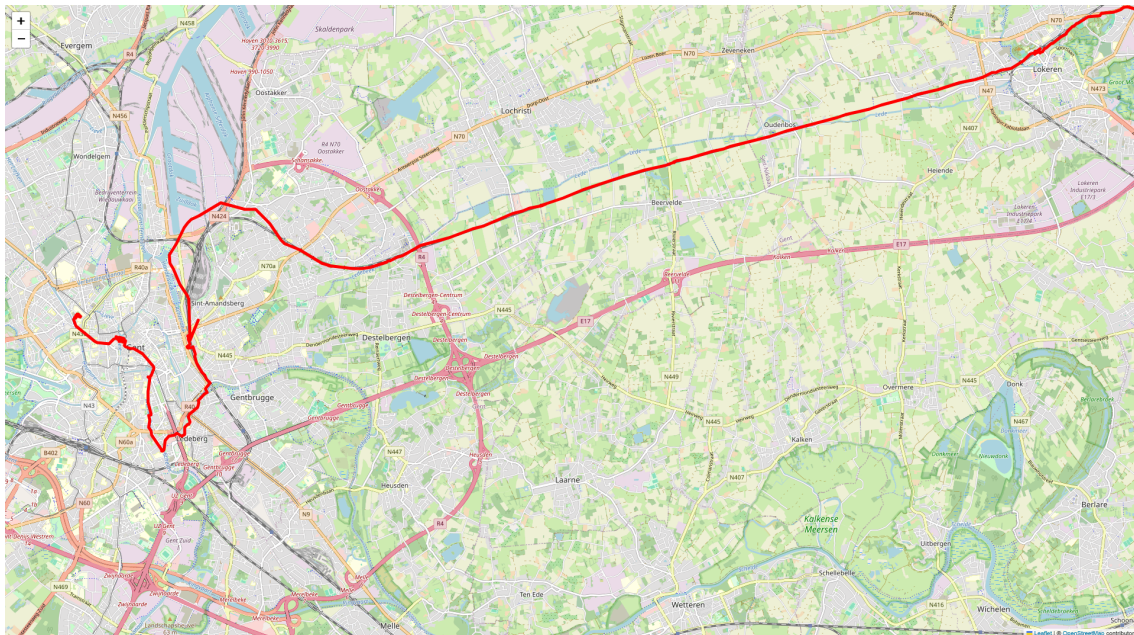


Figure 5.1: Echte pad

5.1.1 Variabelen in kaart brengen

Dan wilden we de 2 parameters van het standaard obfuscatie-algoritme testen. Namelijk de radius van de privacycirkels en de tijdsinterval tussen verzonden punten. De variabelen werden samen met Toon Dehaene en Michiel Willocx zorgvuldig gekozen.

- **Radius(m):** 0, 50, 100, 200, 400, 800, 1600
- **Tijdsinterval(s):** 0, 60, 300, 600, 1800, 3600

Deze variabelen resulteren in 42 mogelijke combinaties. De beschreven route werd daarom 11 keer afgelegd met 4 verschillende telefoons. In Figuur 5.2 wordt het effect van een privacyradius van 0, 400 en 1600 meter geïllustreerd. Omdat het tijdsinterval geen invloed heeft op het genereren van nieuwe privacycirkels, kan deze parameter afzonderlijk worden geëvalueerd. In de afbeeldingen zien we dat een radius van 0 meter de originele route exact volgt. Bij een privacyradius van 400 meter kunnen we aanzienlijke overlap tussen bepaalde cirkels zien, wat het gevolg is van de willekeurige plaatsing van nieuwe cirkels. Bij een radius van 1600 meter kunnen we zien dat slechts vier cirkels volstaan om bijna volledig klein Gent te bedekken.

In Figuur 5.3 wordt het effect van verschillende tijdsintervallen weergegeven. Deze afbeeldingen zijn gemaakt met een privacyradius van 0 meter, zodat enkel de invloed van het tijdsinterval zichtbaar is. De geobfusceerde locatie verschuift afhankelijk van waar de privacycirkels liggen. Uit de resultaten blijkt dat een tijdsinterval van 0 seconden ervoor zorgt dat het originele pad behouden blijft. Bij een tijdsinterval van 5 minuten is er meer detail zichtbaar rond de Korenmarkt, waar een langdurige stop plaatsvond. De rest van het pad is minder

gedetailleerd, maar geeft nog steeds een algemeen beeld van de afgelegde route. Met een tijdsinterval van een uur zijn er slechts twee punten in Gent, waardoor er weinig nuttige informatie over het exacte traject beschikbaar is, behalve dat de campus en de Korenmarkt werden bezocht.

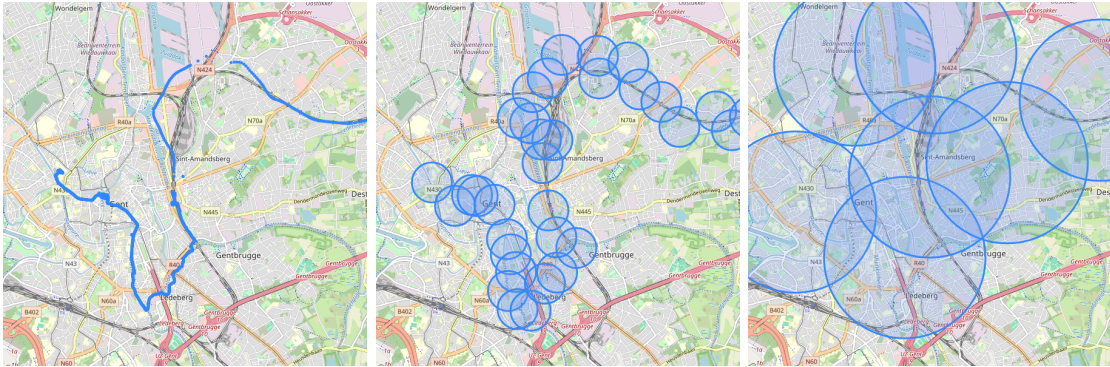


Figure 5.2: Vergelijking grootte privacycirkel

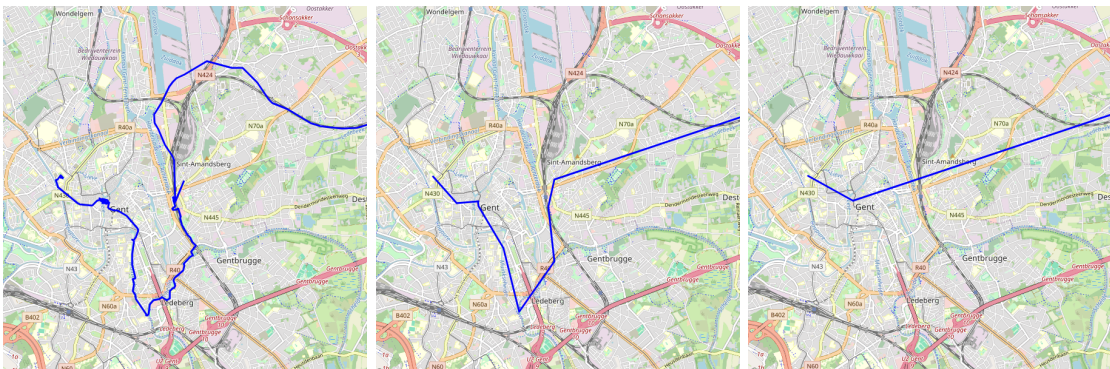


Figure 5.3: Vergelijking tijdsinterval locatiegegevens

5.1.2 Uiteindelijke resultaat

Dan voor alles samen te testen hebben we 1 trace genomen die het meest accuraat was naar de echte beweging van de route. Deze hebben we dan met het algoritme dat lokaal loopt in de library voor de 42 variabelen combinaties laten berekenen. Al deze combinaties zijn te veel voor op deze thesis maar zijn te vinden op deze github page om zo overzichtelijker te kunnen vergelijken. De gekozen visualisatie is te zien in deze Figuur 5.4. De gekozen variabelen voor deze visualisatie is een privacycirkel met een straal van 400 meter en een tijds interval van 5 minuten.

Op deze afbeeldingen kunnen we het volgende zien.

- Er zijn slechts privacy cirkels waar het echte pad langs is geweest. Dit is doordat de permanente opslag tussen iedere test wordt verwijderd. In het echt zou de kaart veel

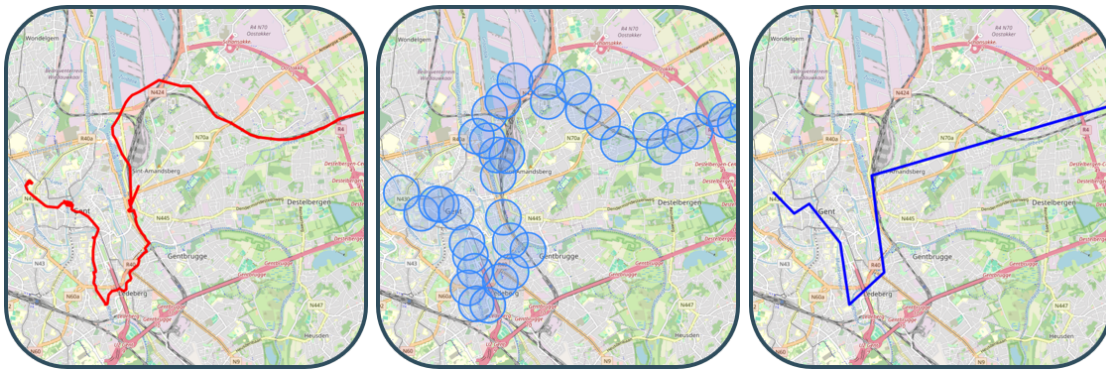


Figure 5.4: Obfuscatie-algoritme gevisualiseerd

voller staan en minder 1 pad duidelijk zijn. Dit is belangrijk moest de data op het apparaat toch gecomprimeerd worden omdat uit deze cirkels geen pad gemaakt kan worden.

- Aan gezien het interval 5 minuten is niet iedere privacycirkel met elkaar verbonden. Bij een interval van 0 seconden is dit wel het geval.
- We kunnen ook duidelijk zien dat snelheid een grote invloed heeft op de uiteindelijke trace. Zo zijn de delen die met de fiets afgelegd zijn kortere lijnstukken. En aangezien de trein rit van Gent-Dampoort naar Lokeren slechts 12 minuten duurt is het logisch dat dit het langste lijnstuk is.

6

Evaluatie en discussie

Het voorgestelde systeem wordt geëvalueerd aan de hand van de twee centrale onderzoeksvragen: de gebruiksvriendelijkheid en uitbreidbaarheid van de Android-library, en de mate waarin de obfuscatie de privacy van de gebruiker beschermt.

6.1 Evaluatie van de onderzoeksvragen

6.1.1 Gebruiksvriendelijkheid en uitbreidbaarheid

De library is ontworpen met een duidelijke abstractielaag, waardoor ontwikkelaars eenvoudig eigen obfuscatie-algoritmes kunnen implementeren via de `LocationObfuscator`-interface. De standaardinitialisatie vereist slechts enkele regels code dankzij goed gekozen standaardinstellingen, waardoor de instapdrempel laag blijft. Integratie in bestaande projecten verloopt vlot, omdat de library compatibel is met gangbare Android-componenten zoals de `WorkManager` en `FusedLocationProvider`. Hierdoor kunnen ontwikkelaars snel locatieverzameling toevoegen zonder diepgaande kennis van de onderliggende complexiteit.

6.1.2 Effectiviteit van de obfuscatie

De obfuscatie beschermt de locatie van de gebruiker door ruis toe te voegen en privacy-cirkels te hanteren, waardoor individuele sporen niet langer direct aan een specifieke gebruiker kunnen worden gekoppeld. Uit de experimenten blijkt dat het nog steeds mogelijk is om algemene bewegingspatronen af te leiden die nuttig zijn voor marktonderzoek, terwijl

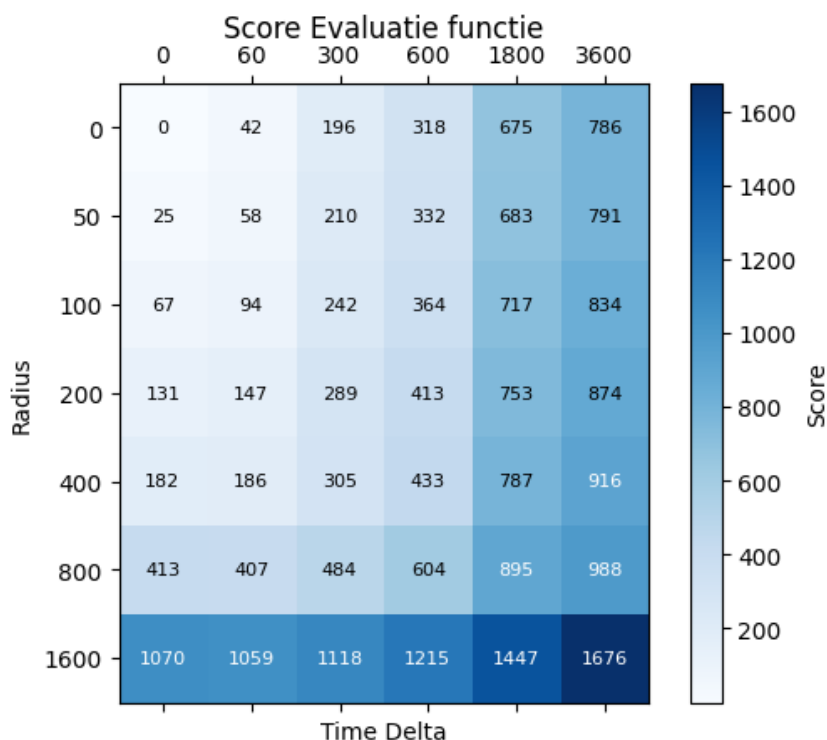


Figure 6.1: Kwantitatieve evaluatieresultaten

gevoelige locaties effectief worden afgeschermd. Het is echter belangrijk dat de variabelen zorgvuldig gekozen worden. De data blijft veilig binnen de applicatie, aangezien de originele, nauwkeurige locatiegegevens het apparaat nooit verlaten of worden opgeslagen. Voor data brokers is de geobfusceerde data minder waardevol voor individuele profilering, maar blijft waarschijnlijk bruikbaar voor geaggregeerd onderzoek. Voor een grondige evaluatie hiervan is echter grootschalig onderzoek nodig.

Naast deze kwalitatieve beoordeling is ook een kwantitatieve evaluatie uitgevoerd. Hiervoor werd een evaluatiefunctie ontwikkeld, waarvan de code te vinden is in B. Deze functie werkt als volgt: voor elk punt van de originele route wordt bepaald of het punt is weggelaten. Indien het punt behouden is, wordt de afstand berekend tussen het originele en het geobfusceerde punt. Indien het punt is weggelaten, wordt de minimale afstand genomen tussen (1) het originele punt en het meest recent geobfusceerde punt, of (2) het originele punt en het dichtstbijzijnde geobfusceerde punt in de tijd.

De resultaten hiervan zijn te zien in Figuur 6.1. Dit zijn de gemiddelde afstanden per punt tot het, in tijd, dichtstbijzijnde geobfusceerde punt. Hieruit blijkt duidelijk dat zowel de radius als het tijdsinterval invloed hebben op de mate van obfuscatie. Dit is een evaluatie van de locatie-obfuscatie; een kwantitatieve evaluatie op het vlak van bruikbaarheid is niet mogelijk met de huidige datasets, aangezien het maken van heatmaps veel meer data vereist dan de 42 beschikbare traces.

De library kan voor een breed toepassingsveld gebruikt worden. Voor applicaties zoals Strava, waar locatie juist accuraat moet zijn, kan de library als facade gebruikt worden of kan de ontwikkelaar een locatie-obfuscator schrijven die enkel bepaalde locaties obfusceert (zoals een thuisadres). Door de hoge instelbaarheid van de library is dit een vooruitgang ten opzichte van enkel het coarse permissiesysteem van Android, waar de developer niets kan aanpassen aan de toepassing. Waar locatie minder van belang is en infrequent nodig is, kan dit door de frequentie van locatie-updates laag te zetten en met de background task infrequent en kort te laten lopen.

Dit alles zonder dat de developer een Worker, permissies of de FusedLocationProvider hoeft te begrijpen. Zo kan er uitsluitend gefocust worden op het obfuscatie-algoritme in locatieprivacy-onderzoek, of kan bij strakke deadlines de leercurve beperkt blijven.

6.1.3 Beperkingen

Ondanks alle maatregelen hebben de library en de testapplicatie enkele beperkingen:

- In de huidige versie van de library is slechts beperkt onderzoek verricht naar de langdurige stabiliteit van locatieverzameling in de achtergrond en naar mogelijke onbekende beperkingen die verder reiken dan de maximale threaddtijd van 10 minuten per 15 minuten. Om dit grondig te testen, zouden meerdere langdurige runtimes moeten worden voorzien.
- Voor de testapplicatie is een fallbackmechanisme geïmplementeerd voor mislukte HTTP-verzoeken, waarbij deze tijdelijk worden opgeslagen in de shared preferences van Android. Echter, shared preferences zijn niet geschikt voor het verwerken van grote hoeveelheden data, wat leidt tot vertragingen bij het lezen en schrijven, vooral wanneer er gedurende langere tijd geen netwerkverbinding beschikbaar is. Bovendien kan de plotselinge piek in internetverkeer bij het herstellen van een netwerkverbinding resulteren in crashes op Android. Verdere uitbreiding van de testapplicatie vereist daarom throttlingmechanismen en een robuustere permanente opslagoplossing, met name voor scenario's waarin testapparaten langdurig geen internetverbinding hebben.

6.2 Verder werk

Dit onderwerp zal in de nabije toekomst een bron van verder onderzoek blijven. Enkele mogelijke richtingen voor toekomstig werk zijn:

- Een mogelijke uitbreiding is het aanpassen van de standaardimplementatie van het obfuscatie-algoritme, waarbij de radius van de privacycirkel afhankelijk is van de straatedichtheid en het interval tussen locatiepunten afhankelijk is van de vervoersmethode.

- Standaardimplementaties (presets) voor bepaalde doeleinden implementeren, zodat minder configuratie nodig is als de developer andere doeleinden heeft dan de standaardinstelling.
- Verder kunnen deze standaardimplementaties ook rekening houden met de toestemmingen die de applicatie heeft verkregen. Er is namelijk geen sterke obfuscatie nodig bovenop coarse locaties van Android.
- Een grootschalige test met vele gebruikers om de bruikbaarheid voor onderzoek naar massabewegingen te kunnen testen.

6.3 Gebruik van de library

6.3.1 Initialisatie

Het gebruik van de library verloopt in enkele duidelijke stappen. Eerst wordt een instantie van de `LocationManager` aangemaakt, waarbij de `MainActivity` als parameter wordt meegegeven. Dit is noodzakelijk om een observer te kunnen koppelen aan de lifecycle van de `MainActivity`. Vervolgens wordt een `LocationObfuscator` ingesteld; dit kan zowel de standaardimplementatie zijn als een door de ontwikkelaar zelf geïmplementeerde klasse. Ten slotte wordt een callbackfunctie geregistreerd, die door de library wordt aangeroepen zodra er een nieuwe locatie beschikbaar is.

```
class MainActivity : ComponentActivity() {
    private val locationManager = LocationManager.getInstance(this)
    init {
        locationManager.locationObfuscator =
            ↪ LocationObfuscatorV1.getInstance()
        locationManager.callback =
            { location -> sendLocationToServer(location) }
        locationManager.startTracking()
    }
}
```

6.3.2 LocationObfuscator zelf implementeren

Om een eigen locatie-obfuscator te implementeren, kan men onderstaande aanpak volgen. De eenvoudigste implementatie bestaat erin dat de locatie verkregen via de `FusedLocationProvider` als argument wordt doorgegeven aan de `obfuscateLocation`-functie, die vervolgens een tuple van coördinaten en tijdstempel retourneert. De overige drie functies (`load`, `store` en `clearStorage`) zijn bedoeld voor het beheren van permanente opslag. Indien nodig kun-

nen variabelen via de constructor worden meegegeven. Het filesDir-object dat aan deze functies wordt doorgegeven, biedt toegang tot de opslagruimte van de applicatie.

```
import android.location.Location
import be.kuleuven.milanschollier.safegps.LatLonTs
import be.kuleuven.milanschollier.safegps.LocationObfuscator
import java.io.File
class ImplementationExample private constructor() : LocationObfuscator {
    override fun obfuscateLocation(location: Location): LatLonTs {
        return LatLonTs(location.latitude, location.longitude,
            ↪ location.time)
    }
    override fun load(filesDir: File) { return }
    override fun store(filesDir: File) { return }
    override fun clearStorage(filesDir: File) { return }
}
```

6.3.3 Live data

De developer kan gebruikmaken van live data in de library. Zo zijn finePermission, coarsePermission en running live data, waarmee de applicatie intuïtief kan reageren op de werking van de library.

```
val finePermission = locationManager.finePermission.observeAsState()
Button(
    onClick = {
        locationManager.getFinePermission(true)
    },
    modifier = Modifier
        .weight(1f)
        .fillMaxSize(),
    enabled = finePermission.value == false
) {
    Text(text = if (finePermission.value == true) "Fine permission
        ↪ granted" else "Ask Fine Permission")
}
```


7

Conclusie

Deze thesis demonstreert dat het mogelijk is om locatiegegevens on-device te obfusceren zonder de bruikbaarheid voor analyse significant aan te tasten. Door het ontwikkelen van een Android-library die locaties en tijdstempels vervormt via ruis en privacycirkels, wordt voorkomen dat individuele gebruikers geïdentificeerd kunnen worden via hun bewegingspatroon. De kern van de oplossing ligt in het behoud van data-utility: geaggregeerde inzichten, zoals mobiliteitstrends of drukteanalyses, blijven mogelijk, terwijl gevoelige individuele informatie wordt beschermd.

Een belangrijk succes is de volledige onafhankelijkheid van externe servers. Alle verwerking vindt plaats op het apparaat zelf, waardoor de originele, nauwkeurige locatie nooit het toestel verlaat. Dit elimineert het risico op datalekken bij overdracht of opslag. Technisch werd dit gerealiseerd door een modulair ontwerp, waarbij de library naadloos integreert met Android's Fused Location Provider en WorkManager voor betrouwbare achtergrondverwerking. Daarnaast biedt de implementatie van een standaard obfuscatie-algoritme op basis van privacycirkels een evenwicht tussen anonymisering en nauwkeurigheid, geïnspireerd op eerder onderzoek.

De library verlaagt niet alleen de implementatiedrempel voor ontwikkelaars door complexe privacytechnieken te abstraheren, maar draagt ook bij aan een paradigmaverschuiving in databeheer. In plaats van privacy als een afterthought te behandelen, wordt bescherming een inherent onderdeel van het ontwerp. Dit sluit aan bij groeiende regelgevende eisen en maatschappelijke vraag naar transparante datapraktijken.

Hoewel verdere optimalisatie mogelijk blijft, toont deze aanpak aan dat technische oplossingen een sleutelrol kunnen spelen in het verzoenen van data-innovatie en individuele rechten.

Door privacy-by-design toegankelijk te maken, zet deze thesis een stap richting een toekomst waarin technologie niet ten koste gaat van gebruikerscontrole, maar deze net versterkt.

Bibliography

- [1] A. Stiru, "Hack bij locatiedatabroker gravity analytics treft miljoenen smartphonegebruikers", 2025. [Online]. Available: <https://tweakers.net/nieuws/230770/hack-bij-locatiedatabroker-gravity-analytics-treft-miljoenen-smartphonegebruikers.html>.
- [2] A. Bannister, 2021. [Online]. Available: <https://portswigger.net/daily-swig/trilateration-vulnerability-in-dating-app-bumble-leaked-users-exact-location>.
- [3] C. Newton, 2019. [Online]. Available: <https://www.theverge.com/interface/2019/3/28/18285274/grindr-national-security-cfius-china-kunlun-military>.
- [4] D. S. Foundation. "Django web framework". [Online]. Available: <https://www.djangoproject.com/>.
- [5] A. Developers. "Android activity lifecycle". [Online]. Available: <https://developer.android.com/guide/components/activities/activity-lifecycle>.
- [6] A. Developers. "Decision tree background work". [Online]. Available: <https://developer.android.com/develop/background-work/background-tasks#user-initiated>.
- [7] A. Developers. "Android alarmmanager documentation". [Online]. Available: <https://developer.android.com/develop/background-work/services/alarms/schedule>.
- [8] A. Developers. "Persistent background tasks in android". [Online]. Available: <https://developer.android.com/develop/background-work/background-tasks/persistent>.
- [9] A. Developers. "Kotlin coroutines guide". [Online]. Available: <https://developer.android.com/kotlin/coroutines>.
- [10] K. D. Boeck, "Coconut project", Ph.D. dissertation, KULeuven, 2023.
- [11] T. Dehaene, M. Willocx, B. Lagaisse, and V. Naessens, "Masking location streams in the presence of colluding service providers", eng, in *2023 IEEE European Symposium on Security and Privacy Workshops (EuroSPW)*, IEEE, 2023, pp. 405–414, isbn: 9798350327205.
- [12] A. Developers. "Background location and battery life". [Online]. Available: <https://developer.android.com/develop/sensors-and-location/location/battery>.
- [13] R. Guru. "Singleton design pattern". [Online]. Available: <https://refactoring.guru/design-patterns/singleton>.
- [14] A. Developers. "Access app-specific files". [Online]. Available: <https://developer.android.com/training/data-storage/app-specific>.
- [15] P. S. Theme. "Folium library for geospatial visualization". [Online]. Available: <https://python-visualization.github.io/folium/latest/>.



Code van het obfuscatie algoritme

```
override fun obfuscateLocation(location: Location): LatLonTs? {
    val lat = location.latitude
    val lon = location.longitude
    val ts: Long = location.time

    val matchedReports = historyBlobs.filter { report ->
        Geodesic.WGS84.Inverse(
            report.first,
            report.second,
            lat,
            lon,
            GeodesicMask.DISTANCE
        ).s12 < report.third
    }

    val blob: PrivacyBlob
    if (matchedReports.isEmpty()) {
        val sampledRadius = radiusSampler()
        val sampledAngle = Math.toDegrees(angleSampler())
        val res = Geodesic.WGS84.Direct(lat, lon, sampledAngle,
            ↪ sampledRadius)
        blob = PrivacyBlob(res.lat2, res.lon2, _blobRadius)
        println("Generated new privacy blob: $blob")
    }
}
```

```

        historyBlobs.add(blob)
    } else {
        blob = matchedReports.minBy { report ->
            Geodesic.WGS84.Inverse(
                report.first,
                report.second,
                lat,
                lon,
                GeodesicMask.DISTANCE
            ).s12
        }
    }

    val lastLocation = perturbedLocationCache.lastOrNull() ?:
    ↪ LatLonTs(0.0, 0.0, 0)

    if (ts - lastLocation.third < _deltaTime * 1000) {
        return null
    }

    //Voor jitter tegen te gaan als in de buurt van vorige locatie die
    //privacy blob doorsturen ongeacht andere dichterbij is
    val res: LatLonTs = if (Geodesic.WGS84.Inverse(
        lastLocation.first,
        lastLocation.second,
        lat,
        lon,
        GeodesicMask.DISTANCE
    ).s12 < _blobRadius
    ) {
        LatLonTs(lastLocation.first, lastLocation.second, ts)
    } else {
        LatLonTs(blob.first, blob.second, ts)
    }

    perturbedLocationCache.add(res)
    if (perturbedLocationCache.size > 5)
    ↪ perturbedLocationCache.removeAt(0)
    return res
}

```

B

Code van de evaluatie functie

```
fn calc_score(_original: &Vec<LatLonTs>, _obfuscated: &Vec<LatLonTs>) ->
↳ f64 {
    let mut distance: f64 = 0.0;
    for orig in _original {
        // Zoek alle obfuscated punten met dezelfde timestamp
        if let Some(obf) = _obfuscated.iter().find(|o| o.ts == orig.ts) {
            let p1 = Point::from((orig.lon, orig.lat));
            let p2 = Point::from((obf.lon, obf.lat));
            distance += Geodesic::distance(p1, p2);
        } else {
            // Zoek de twee obfuscated punten met ts net voor en na
            ↳ orig.ts
            let mut before: Option<&LatLonTs> = None;
            let mut after: Option<&LatLonTs> = None;
            for obf in _obfuscated {
                if obf.ts <= orig.ts {
                    if before.is_none() || obf.ts >
↳ before.as_ref().unwrap().ts {
                        before = Some(obf);
                    }
                }
            }
            if obf.ts >= orig.ts {
```

```

        if after.is_none() || obf.ts <
            ↪ after.as_ref().unwrap().ts {
                after = Some(obf);
            }
    }
}
let p_orig = Point::from((orig.lon, orig.lat));
let d_before = before.map(|b| Geodesic::distance(p_orig,
    ↪ Point::from((b.lon, b.lat))));
let d_after = after.map(|a| Geodesic::distance(p_orig,
    ↪ Point::from((a.lon, a.lat))));
match (d_before, d_after) {
    (Some(d1), Some(d2)) => distance += d1.min(d2),
    (Some(d1), None) => distance += d1,
    (None, Some(d2)) => distance += d2,
    (None, None) => {}
}
}
}
if !_original.is_empty() {
    distance /= _original.len() as f64;
}
return distance;
}

```



Beschrijving van deze masterproef in de vorm van
een wetenschappelijk artikel

Development of a smartphone SDK for obfuscating location data

Milan Schollier
Faculty of Engineering Technology
KuLeuven
Ghent, Belgium

Prof. Dr. Vincent Naessens
Faculty of Engineering Technology
KuLeuven
Ghent, Belgium

Dr. Ing. Michiel Willocx
Faculty of Engineering Technology
KuLeuven
Ghent, Belgium

Abstract—In today’s data-driven economy, location data is of immense value. It is used to gain meaningful crowd insight and optimize various sectors of society, ranging from tourism to retail. However, the methods used to collect these valuable location traces often occur without the user’s explicit informed consent. The high precision of location data and the frequency of reports pose significant privacy risks to individuals. These risks include not only the inference of sensitive locations, such as home, work, and leisure activities, but also the construction of social graphs. In the hands of malicious actors, this information could lead to blackmail, coercion, or hate crimes. To address these privacy concerns, this thesis proposes a privacy-friendly alternative to collect valuable location traces. A novel Android library is developed that obfuscates both locations and timestamps by introducing noise, ensuring that individual traces can no longer be linked to a specific user. This prevents sensitive information from leaking while preserving the utility of the data. Importantly, the library operates entirely on-device, ensuring that the real, accurate traces never leave the device. To achieve this, this thesis explores the Android ecosystem on lifecycles and background tasks to design and implement advanced location obfuscation algorithms within the library. A set of experiments are conducted to demonstrate the effectiveness of the solution. Among other evaluations, this thesis compares the proposed library to the Approximate Location feature introduced in Android 12, offering information on advances in mobile privacy measures.

Index Terms—Android library, location obfuscation, location tracking

I. INTRODUCTION

A. Context and problem definition

The central problem is that the real location of a user is usually sent directly to the server and stored there without any form of obfuscation. As a result, location data remains available in its original and unobfuscated form, entails significant privacy risks. Furthermore, this data is often sold on to data brokers without further processing.

In order to guarantee the privacy of users, several conditions must be met. First, data transmission to the server must be secured against interception. Second, the server itself must be adequately protected against data leaks. In addition, it is essential that stored location data is stored in an obfuscated manner and that any resale of this data also takes place only in obfuscated form. Finally, strict control over the parties that gain access to this data is necessary to prevent misuse.

The presence of these potential vulnerabilities makes it difficult for users to have confidence in the current processing of location data.

In addition, the steep learning curve of relevant Android technologies poses an additional challenge. This can lead to errors or security gaps in the implementation, especially with limited experience or tight deadlines. A well-designed library that reduces complexity for developers and provides robust security measures as standard can significantly reduce these risks.

II. RELATED WORK

A. Location Privacy Risks

The collection of high-resolution and high-frequency location data introduces significant privacy risks. Research by Kevin De Boeck has demonstrated that seemingly anonymized datasets can be used to infer private places such as homes, workplaces, and vacation patterns. Even sparse traces can reveal sensitive information, especially in rural areas where few streets exist within a given radius. Malicious actors may use this data for profiling, blackmail, or even targeted crimes. [2]

These risks are not purely theoretical. In practice, data brokers like Gravy Analytics have experienced massive data breaches that exposed millions of users’ location information [1]. Additionally, location-based dating apps have historically leaked users’ exact positions by allowing triangulation through proximity features [4]. As a result, government and military bodies have flagged such applications as national security concerns.

B. Existing Solutions

To mitigate these risks, several obfuscation strategies have been proposed and implemented. Android 12 introduced the concept of “Approximate Location,” allowing users to restrict apps to a coarser location estimate. However, the mechanism used is closed-source, non-configurable by developers, and offers limited transparency.

Academic work has proposed more robust, configurable solutions. One such approach is the concept of “privacy circles,” used in prior work by Toon Dehaene. His system introduced random displacement within a radius and ensured

that location points could not be averaged to recover the original coordinates. His implementation in Rust allowed for batch obfuscation on large datasets but required off-device processing. [3]

This thesis builds upon such prior work by implementing the obfuscation pipeline directly on Android devices, ensuring that accurate location data never leaves the device and that obfuscation is integrated seamlessly into the data collection process.

III. SYSTEM DESIGN

A. Requirements

The design of the Android SDK is guided by both technical and privacy requirements. Functionally, the SDK must:

- Collect location and timestamp data at configurable intervals.
- Apply a robust obfuscation method before any transmission or storage.
- Respect Android's lifecycle, background limitations, and permission model.

From a developer usability perspective, the SDK should:

- Offer a simple integration flow with minimal lines of code.
- Allow developers to plug in their own obfuscation logic.
- Provide debug tools for validation of both obfuscated and raw traces.
- Automatically handle permission checks and WorkManager setup.

B. Architecture

The system consists of three main components:

- **LocationManager:** A singleton [5] responsible for configuration, lifecycle management, and coordination between services.
- **LocationObfuscator:** A WorkManager Worker implementation that handles background location polling, either periodically or via foreground services.
- **LocationWorker:** An abstract interface that allows the injection of custom obfuscation algorithms. A default implementation is provided based on privacy circles.

The SDK separates concerns clearly: location retrieval is decoupled from obfuscation logic, and both are decoupled from data transmission. This ensures flexibility, maintainability, and extensibility for future use cases.

IV. IMPLEMENTATION

A. Obfuscation Algorithm

The default obfuscation algorithm adapts Toon Dehaene's privacy circle approach for real-time data streaming. The core logic is:

- If a new location lies within an existing privacy circle, reuse the circle's centre.
- If outside all circles, generate a new circle that's displaced by a random angle and random distance between 0 and the currently used radius.

- Respect a minimum time interval between emissions to avoid over-reporting.
- Prevent jitter by favouring reuse of the most recently active circle over the closest one. Given the point lies within the most recently active circle.

This algorithm minimizes the risk of location recovery via averaging or triangulation. All operations occur locally, and no raw data is retained or transmitted unless in debug mode. Storage of circles is done via Android's `fileDir`, which is sandboxed per application. So only if the system is compromised a data leak could occur.

B. Android-specific challenges

Due to Android's strict limitations on background execution, the SDK uses WorkManager as its backbone. It supports both foreground services (with a visible notification) and periodic jobs (every 15 minutes max, running up to 10 minutes).

Permission handling is implemented using `ActivityResultContracts.RequestPermission`. The library checks and requests permissions for both coarse and fine location access and guides users to system settings when needed.

To prevent memory leaks and lifecycle crashes, the `LocationManager` was made a singleton so the user couldn't have two instances that together could crash the system. All blocking calls are offloaded to thread or the `LocationWorker`. As Android enforces no blocking operations on the main thread.

V. EVALUATION

A. Experimental Setup

To evaluate the SDK, a controlled test route was followed through Ghent. The route included:

- Departing from KU Leuven Technology Campus,
- Stopping at the Korenmarkt (long idle period),
- Passing De Krook (short idle period),
- Boarding a train at Dampoort towards Lokeren.

This route enabled the evaluation of the SDK under different speeds, environments (urban, indoor), and stop durations. Four Android phones were used to record traces, each with a different configuration of variables. For the obfuscation evaluation, 42 configurations were tested (7 radii $\times$ 6 intervals). All traces were visualized using Jupyter notebooks and the Folium mapping library. Obfuscation results were compared against raw traces to evaluate privacy, continuity, and utility.

B. Key Findings

- **Privacy vs Utility Trade-off:** Larger privacy radii (e.g., 1600m) resulted in fewer, broader circles, which significantly improved anonymity but reduced trace granularity. Conversely, smaller radii retained more precise path information at the cost of lower privacy.
- **Hardware Influence:** Indoor locations (e.g., train or buildings) showed more location jitter, but the algorithm's reuse of prior circles prevented excessive data fluctuation.
- **Approximate vs Fine Location:** Android's built-in coarse location showed signs of both stochastic and deterministic

obfuscation, but the proposed SDK offered more consistency and developer control.

These results confirm that on-device obfuscation is feasible and effective without requiring backend involvement, and can be tuned according to the intended level of privacy and analytical utility.

VI. CONCLUSION

This thesis demonstrates that it is possible to obfuscate location data on-device without significantly affecting its usability for analysis. By developing an Android library that distorts locations and timestamps via noise and privacy circles, it is prevented that individual users can be identified via their movement patterns. The core of the solution lies in preserving data utility: aggregated insights, such as mobility trends or crowd analyses, remain possible, while sensitive individual information is protected.

A key success is the complete independence from external servers. All processing takes place on the device itself, so that the original, accurate location never leaves the device. This eliminates the risk of data leakage during transmission or storage. Technically, this was achieved by a modular design, where the library seamlessly integrates with Android's Fused Location Provider and WorkManager for reliable background processing. In addition, the implementation of a standard obfuscation algorithm based on privacy circles provides a balance between anonymization and accuracy, inspired by previous research.

The library not only lowers the implementation barrier for developers by abstracting complex privacy techniques, but also contributes to a paradigm shift in data management. Instead of treating privacy as an afterthought, protection becomes an inherent part of the design. This aligns with growing regulatory requirements and societal demand for transparent data practices.

While further optimization remains possible, this approach shows that technical solutions can play a key role in reconciling data innovation and individual rights. By making privacy-by-design accessible, this thesis takes a step towards a future in which technology does not come at the expense of user control, but rather enhances it.

REFERENCES

- [1] A. Stiru, "Hack bij locatiedatabroker gravity analytics treft miljoenen smartphonegebruikers", 2025. [Online]. Available: <https://tweakers.net/nieuws/230770/hack-bij-locatiedatabroker-gravity-analytics-treft-miljoenen-smartphonegebruikers.html>
- [2] K. D. Boeck, "Coconut project", Ph.D. dissertation, KU Leuven, 2023.
- [3] T. Dehaene, M. Willocx, B. Lagasse, and V. Naessens, "Masking location streams in the presence of colluding service providers", eng, in 2023 IEEE European Symposium on Security and Privacy Workshops (EuroSPW), IEEE, 2023, pp. 405–414, isbn: 9798350327205.
- [4] A. Bannister, 2021. [Online]. Available: <https://portswigger.net/daily-swig/trilateration-vulnerability-in-dating-app-bumble-leaked-users-exact-location>
- [5] R. Guru. "Singleton design pattern". [Online]. Available: <https://refactoring.guru/design-patterns/singleton>

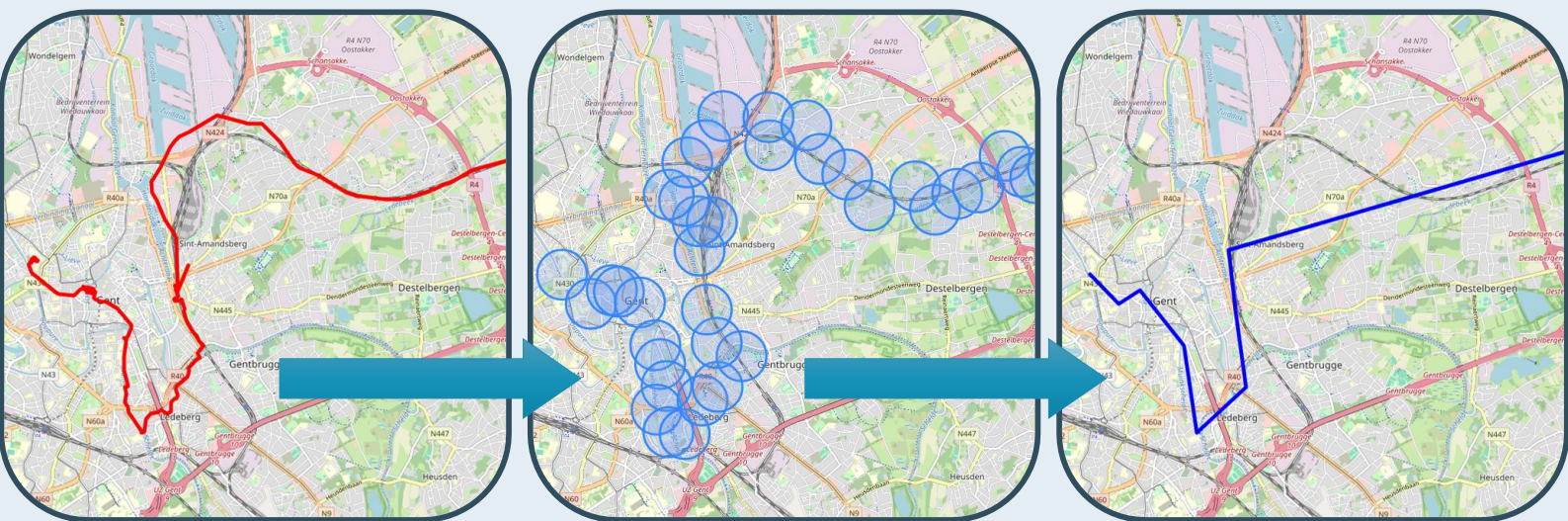
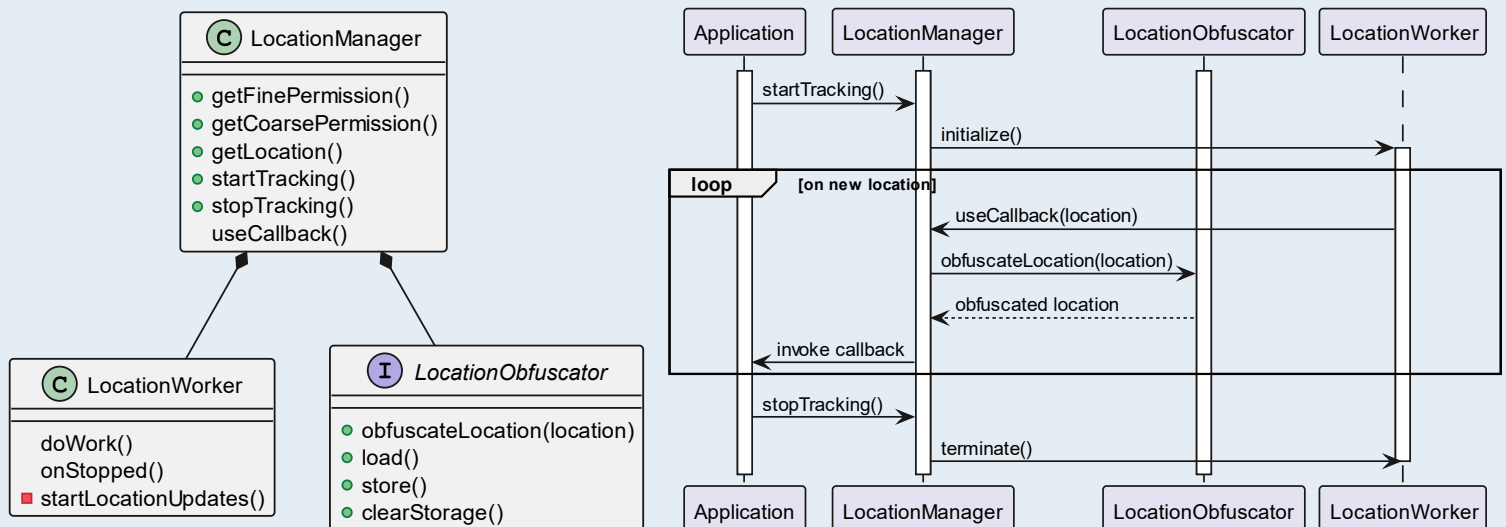
D

Poster

Ontwikkeling van een smartphone sdk voor het obfusceren van locatie data

Abstract

Locatiedata biedt waardevolle inzichten, maar brengt aanzienlijke privacy risico's met zich mee, zoals het afleiden van gevoelige locaties en sociale grafen. Deze thesis introduceert een innovatieve Android-bibliotheek die locatiedata obfusceert door ruis toe te voegen aan locaties en tijdstempels. Dit voorkomt dat individuele sporen aan gebruikers worden gekoppeld, terwijl de bruikbaarheid van de data behouden blijft. De bibliotheek werkt volledig op het apparaat, zodat nauwkeurige gegevens nooit het toestel verlaten. Experimenten tonen de effectiviteit aan en vergelijken de oplossing met Android 12's Approximate Location-functie, wat bijdraagt aan verbeterde mobiele privacy.





GenAI code of conduct for students

Generative AI (GenAI) assistance tools can be used to generate text, image, code, video, music, or combinations of these. It includes typical tools like (but this list is not limited to): ChatGPT, Google Gemini, MS Copilot, Midjourney, Claude.ai, Perplexity.ai, Dall-E, etc.

Student Information

Student name: Milan Schollier

This form is related to my Master thesis.
Title Master thesis: Ontwikkeling van een
smartphone SDK
voor het obfusceren
van locatie data
Promoter: prof. dr. Vincent Naessens
Daily supervisor: dr. ing. Michiel Willocx

Use of GenAI Assistance

Please indicate with "X" (possibly multiple times) in which way you were using GenAI:

☐ I did not use any GenAI assistance tool.

- ☒ I did use GenAI assistance. In this case, specify which ones (e.g., ChatGPT, ...):
Github Copilot

GenAI Assistance Used As/For

- ☐ MS Copilot
- ☒ Language assistance
- ☐ Search engine
- ☐ Literature search
- ☐ Short-form input assistance
- ☐ Generating programming code
- ☐ Generating new research ideas
- ☐ Generating blocks of text
- ☐ Other (specify): _____

Code of Conduct for Different Uses

Language Assistant for Reviewing or Improving Texts

This use is similar to using spelling and grammar check tools, and you do not have to refer to the use of GenAI in the text. Be careful:

- Using GenAI tools on texts you did not write yourself to cover up plagiarism (by paraphrasing original texts) is not allowed.

Search Engine for Information or Existing Research

This use is similar to a Google search or checking Wikipedia. If you then write your own text based on this information, you do not have to refer to the use of GenAI in the text. Be careful:

- Be aware that the output of the GenAI tool cannot be guaranteed as a 100% reliable source of information.
- The output may not be entirely correct and is limited due to the databases it uses. Knowledge evolves and may change over time, and it may be that the database of the GenAI tool is not up to date.

Literature Search

This use is comparable to a Google Scholar search. Be careful:

- Be aware that the output is restricted to the database it is built on. After this initial search, look for scientific sources and conduct your own analysis.
- GenAI tools (like ChatGPT) may output no or wrong references. As a student, you are responsible for further checking and verifying the accuracy of references.

Short-form Input Assistance

This use is similar to Google Docs powered by generative language models.

Generating Programming Code

The use of GenAI for coding should be explicitly allowed by the teacher. If used for coding, correctly mention the use of GenAI assistance and cite it.

Generating New Research Ideas

Further verify whether the idea is novel or not. It is likely that it is related to existing work, which should be referenced.

Generating Blocks of Text

Inserting blocks of text without quotes and a reference to GenAI assistance in your report or thesis is not allowed. Be careful:

- When you literally copy elements from a conversation with a GenAI tool, quote them between quotation marks and refer to them according to the specified reference style.

- Describe the use of the GenAI tool (tool name, version, date, etc.) in the method section and optionally add the full conversation as an attachment.

Other

Contact the professor of the course or the promoter of the thesis. Inform also the program director. Motivate how you comply with Article 84 of the exam regulations. Explain the use and added value of ChatGPT or another AI tool.

Further Important Guidelines and Remarks

- GenAI assistance cannot be used related to data or subjects under Non-Disclosure Agreement.
- GenAI assistance cannot be used related to sensitive or personal data due to privacy issues.
- Take a scientific and critical attitude when interacting with GenAI assistance and interpreting its output.
- As a student, you are responsible for complying with Article 84 of the exam regulations: your report or thesis should reflect your own knowledge, understanding, and skills.

Exam Regulations Article 84

“Every conduct individual students display with which they (partially) inhibit or attempt to inhibit a correct judgement of their own knowledge, understanding and/or skills or those of other students, is considered an irregularity which may result in a suitable penalty. A special type of irregularity is plagiarism, i.e., copying the work (ideas, texts, structures, designs, images, plans, codes, ...) of others or prior personal work in an exact or slightly modified way without adequately acknowledging the sources.”

Additional Reading

More information about being transparent on the use of GenAI assistance and about citing and referencing GenAI can be found on the student website.

A Few Final Words

If you are uncertain whether or not you should declare your use of AI tools, discuss the matter with your teacher or promoter. It is safer to declare AI use when it is not needed than to withhold that declaration when it is required.

Finally, remember that advanced AI tools are new and that they can do things they could not do until recently. It is important to follow up on the most recent developments in AI technologies and communicate openly with your teachers, assistants, supervisors, and peers.

KU LEUVEN - GENT

Faculteit Industriële Ingenieurswetenschappen
Campus Rabot, Gebroeders De Smetstraat 1
9000 Gent, België
tel: + 32 9 265 86 10
iiw.gent@kuleuven.be
www.iw.kuleuven.be

